A structured quasi-Newton augmented Lagrangian algorithm for nonlinear least-squares with linear and nonlinear constraints

Pierre Borie[1*], Fabian Bastin[1*] and Stéphane Dellacherie[2,3]

[1*]Department of Computer Science and Operations Research, Université de Montréal, Montreal, QC, Canada.
[2]Hydro-Québec, Montreal, QC, Canada.
[3]Université du Québec à Montréal, Montreal, QC, Canada.

*Corresponding author(s). E-mail(s): pierre.borie@umontreal.ca; bastin@iro.umontreal.ca;
Contributing authors: dellacherie.stephane@hydroquebec.com;

Abstract

We present an algorithm for solving nonlinear least-squares problems subject to a mix of nonlinear constraints, linear equalities, and bounds. The nonlinear constraints are handled through an augmented Lagrangian reformulation of the objective, while the linear constraints are kept explicit. Each outer iteration therefore requires the approximate minimization of the augmented Lagrangian over a polyhedron, which we carry out with a gradient projection technique combined with a trust-region step computation. To capture the second-order information specific to the least-squares structure, we approximate the Hessian of the augmented Lagrangian by a Gauss–Newton term corrected by a structured quasi-Newton update, activated by a hybrid switching rule. The correction is built from a secant equation that we derive for the augmented Lagrangian itself. The outer iteration follows the augmented Lagrangian framework of Conn, Gould, Sartenaer and Toint for linearly constrained subproblems, for which we provide an explicit inner solver: a trust-region method whose Cauchy point is computed along nested faces of the polyhedron, without any projection onto it, and safeguarded by classical active-set devices, namely the release of bounds based on least-squares multiplier estimates and ε-active bounds. We prove that this inner solver satisfies the stopping rule required by the outer analysis, which yields global convergence of the overall method to a first-order critical point, and report numerical experiments. Our method outperforms an augmented Lagrangian solver built on the same outer iteration but without least-squares

structure, and is competitive in computation time with a state-of-the-art solver for nonlinear programming.

Keywords: constrained nonlinear least-squares, augmented Lagrangian method, Julia programming language, structured quasi-Newton update

1 Introduction

We consider the constrained nonlinear least-squares (NLS) problem

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & \frac{1}{2}\|r(x)\|^2 \\ \text{s.t.} \quad & c(x) = 0 \\ & x \in \Omega, \end{aligned} \tag{1}$$

where $\|\cdot\|$ denotes the ℓ_2 norm. The residual and constraint functions $r\colon \mathbb{R}^n \to \mathbb{R}^{n_r}$ and $c\colon \mathbb{R}^n \to \mathbb{R}^{n_c}$ are assumed to be twice continuously differentiable with respective Jacobians J and C. We may also write the least-squares objective as a general function f with gradient

$$\nabla f(x) = J(x)^\top r(x).$$

The set Ω consists of linear constraints and is defined as

$$\Omega = \{x \in \mathbb{R}^n \mid Ax = b,\ l \leq x \leq u\}, \tag{2}$$

with $A \in \mathbb{R}^{m \times n}$, $m < n$, $b \in \mathbb{R}^m$, and $l, u \in \mathbb{R}^n$. Without loss of generality, some components of the latter two vectors can be set to $\pm\infty$ for unbounded parameters. For convenience, we also assume that the linear equalities do not involve single-variable constraints. Formulation (1) is also suitable for problems with nonlinear inequality constraints of the form $g(x) \geq 0$. The latter are transformed into equality constraints by adding non-negative slack variables, which gives the new constraints

$$g(x) - \nu = 0, \quad \nu \geq 0.$$

The lower and upper bounds associated with these slack variables are thus 0 and ∞ respectively. Linear inequality constraints can be converted into equalities similarly.

The Lagrangian for problem (1), with respect to the nonlinear constraints, is defined by

$$\mathcal{L}(x, \lambda) = f(x) + \langle \lambda, c(x) \rangle,$$

where $\lambda \in \mathbb{R}^{n_c}$ is the vector of Lagrange multipliers and $\langle \cdot, \cdot \rangle$ is the standard inner product.

An algorithm for solving problem (1) aims to find a first-order critical point (x_*, λ_*) that satisfies the necessary condition

$$\begin{aligned} &x_* \in \Omega \text{ and } c(x_*) = 0 \\ &P_\Omega\left[x_* - \nabla_x \mathcal{L}(x_*, \lambda_*)\right] = x_*, \end{aligned}$$

where $P_\Omega[\cdot]$ is the projection operator onto Ω, defined, for any vector x, as

$$P_\Omega[x] = \operatorname*{argmin}_{v \in \Omega} \ \|v - x\|^2.$$

The NLS structure arises in data fitting, parameter estimation, and scientific computing, and its efficient resolution has attracted a sustained research effort [1–3]. A distinctive feature of NLS problems is that the Hessian of the objective decomposes into a first-order part $J(x)^\top J(x)$, where $J(x)$ is the Jacobian of r, and a second-order part involving the individual Hessians of the residuals weighted by their values. Exploiting this decomposition is the central challenge in designing efficient algorithms. The Gauss–Newton (GN) method retains only the first-order term, which is effective when residuals are small at the solution but can lead to poor convergence otherwise. The Levenberg–Marquardt (LM) method, initiated by Levenberg [4] and Marquardt [5] and further studied, for instance, in [6, 7], regularizes the GN matrix by a scalar multiple of the identity, providing robustness at the cost of potentially slower convergence on large-residual problems. A richer class of approaches, known as structured quasi-Newton (SQN) methods, approximates the second-order part through secant equations tailored to the least-squares structure. The NL2SOL package [8, 9] is a popular implementation of this idea, complementing the GN term with a structured quasi-Newton correction updated via a dedicated secant equation. This algorithm also switches between the GN model and the full Hessian depending on the quality of the step. Other hybrid strategies with a switching criterion based on the residual magnitude have also been explored in [10–12]. Further SQN formulations, including factorized updates and sizing strategies, have been proposed in [13–17]. A comprehensive numerical comparison of these techniques can be found in [18].

Extending structured NLS methods to the constrained setting has received comparatively less attention, though it remains an active area of research. Such formulations arise, for example, in data-fitting problems where the constraints enforce the physical consistency of an underlying model [19–21]. The convex case, especially the bound-constrained case, is studied in [22–25].

For more general nonlinear constraints, most methods operate within a sequential quadratic programming (SQP) framework. In that paradigm, each iteration solves a quadratic subproblem obtained by linearizing the constraints and approximating the Hessian of the Lagrangian. An SQP method whose Hessian approximation is based on a secant equation derived from that of Biggs [13] is proposed by Li et al. [26]. A BFGS update of the projected Hessian of a merit function is used in [27], while a structured approximation of the projected Hessian is updated within the null space of the active constraints in [28]. For problems with equality constraints only, more recent work includes regularized approaches. A Levenberg–Marquardt approximation of the

Lagrangian Hessian is combined with a trust-region method in [7], and a regularized KKT system is derived in [29] by adding primal and dual regularization terms. Structured approximations of the Hessian within general constrained formulations have also been studied from a theoretical perspective in [30]. When the constraints can be split into two categories, as in formulation (1), an alternative to SQP methods is the augmented Lagrangian (AL) framework, also known as the method of multipliers, independently introduced by Hestenes [31] and Powell [32], with further theoretical foundations provided by Rockafellar [33]. Rather than solving a sequence of quadratic subproblems, AL methods incorporate the nonlinear constraints into the objective as a penalty term and solve a sequence of subproblems in which only the easier constraints remain. For problem (1), this amounts to approximately minimizing, at each outer iteration, the AL function over the polyhedral set Ω, thereby reducing the problem to a linearly constrained one. This strategy is particularly attractive as the linear part can be handled directly and efficiently by gradient projection techniques, without sacrificing the ability to enforce nonlinear constraints. The theoretical underpinnings of this approach, including global convergence results, are developed in [34–37] and summarized in [38, Chapter 14]. A major implementation of this philosophy is the LANCELOT solver [39], which handles bound-constrained subproblems using a gradient projection technique [40]. More recent AL implementations have broadened this framework in several directions. Convergence is established under weaker constraint qualifications in [41]. A primal-dual AL method that behaves like a stabilized SQP method in the local regime is proposed in [42], a matrix-free AL approach is implemented in [43], and an adaptive AL algorithm with inexact subproblem solves is developed in [44]. For the inner minimization that arises at each outer AL iteration, gradient projection methods [45, 46] are particularly effective for bound-constrained problems as they allow for rapid changes in the active set throughout the iterations. The linear equality case can be handled efficiently, as the projections can be computed by solving a system of normal equations [47]. When both bounds and linear equalities are present, as in our setting, direct projection onto Ω is not available in closed form. Nevertheless, we will see in the next section that the norm of the reduced gradient, complemented by a sign check on least-squares estimates of the bound multipliers as in active-set solvers such as MINOS [48] and LSNNO [49], provides a practical criticality measure, as already noted in [37, Section 7.2].

It is worth pointing out what this combination leaves uncovered in the existing literature. Structured quasi-Newton methods for least-squares are, with the exceptions discussed above, developed for the unconstrained case. Augmented Lagrangian solvers such as LANCELOT [39] and Percival [43] handle bound-constrained subproblems, the other linear constraints being incorporated into the augmented Lagrangian, and treat the objective as a generic nonlinear function. The framework of [37] keeps the linear constraints explicit in the subproblems, but leaves their solution to existing linearly constrained solvers and does not exploit a least-squares structure. The structured methods available for constrained least-squares operate within an SQP framework [26–28] and do not keep bounds and linear equalities explicit in their subproblems. Among the methods surveyed above, we are not aware of one combining the three features at once, namely an approximation exploiting the least-squares structure, a feasible

set mixing nonlinear equalities with a general polyhedron whose linear constraints are never penalized nor linearized, and a supporting global convergence analysis. It is this combination that our algorithm addresses.

The present work brings the structured quasi-Newton philosophy of nonlinear least-squares into the augmented Lagrangian framework. Our contributions are the following.

(i) We derive a structured secant equation for the Hessian of the AL, given in (31), which simultaneously captures the curvature carried by the residuals and that carried by the nonlinear constraints, the latter being weighted by the shifted multipliers $\bar{\lambda}$ produced by the outer iteration. Correcting the Gauss–Newton term of the AL by a structured update satisfying this equation yields an approximation that satisfies the standard secant equation up to second-order terms in the step, the equation holding exactly if the residuals and constraints are affine. To the best of our knowledge, this is the first structured quasi-Newton approximation designed for the AL of a least-squares problem. We complement it with a hybrid switching rule driven by the relative decrease of the AL itself, rather than of the objective, so that the decision to activate the second-order correction reflects the penalized function that is actually minimized.

(ii) We provide an explicit inner solver for the subproblems over the polyhedron (2), in which bounds and linear equalities coexist, extending the gradient projection approach of [40] that underlies the bound-constrained kernel of LANCELOT. The analysis of [37] leaves the solution of these subproblems to existing linearly constrained solvers, the generalized Cauchy point procedure of [36] requires the approximate solution of a linear program at every trial step, and the projected gradient methods of [50] require projections onto Ω. Our Cauchy point is computed along nested faces of Ω, with projections onto subspaces only. Such a path does not provide a Cauchy decrease by itself: we give an example in which the decrease of the model is arbitrarily small compared with any criticality measure, the trust-region analogue of the zigzagging phenomenon of feasible direction methods [51, 52]. We safeguard it with two classical devices, namely the release of the bounds whose least-squares multiplier estimates have the wrong sign, as in gradient projection methods since [53], and ε-active bounds, which correspond to the anti-zigzagging device of [51], the ε-active sets of [54, 55] and the dominant constraints of [37]. We prove the resulting decrease guarantee and the termination of the inner iterations (Section 3.1 and Appendix A).

(iii) We establish global convergence of the resulting method to a first-order critical point of problem (1). Our problem being a particular case of the one considered in [37], the analysis of the outer iterations follows theirs, with their criticality test. Our contribution there is to show that the measure used in our implementation, based on the reduced gradient and on least-squares multiplier estimates, certifies this test without any assumption on the signs of the bound multipliers (Lemma 10, a reformulation of [37, Section 7.2]), and that the inner solver of Section 3.1 satisfies it under a growth condition on the Hessian model only (Assumption 6), which covers every variant we implement.

(iv) We provide TRAULLS, an open-source Julia [56] implementation available at https://github.com/UncertainLab/Traulls.jl. All the experiments of Section 4 can be reproduced from the `benchmark` folder of that repository, in the version tagged `coap-2026`.

Together, these ingredients yield a solver specifically designed for NLS problems while handling constraints of general form, including nonlinear equalities and inequalities alongside linear equalities and bounds.

We end this section by introducing some notation. Components of a vector $x \in \mathbb{R}^n$ are denoted x_i for $i = 1, \ldots, n$. Vectors e_i denote the columns of the $n \times n$ identity matrix written I_n – or simply I when the dimension is clear from the context. When describing iterative processes to solve problem (1), we index vectors and matrices by the iteration number. Our method has a nested structure (Section 2): we use an uppercase index K for its outer iterations and a lowercase index k for the inner iterations. For instance, at outer iteration K, (x_K, λ_K) is the current primal-dual iterate, while for functions evaluated at an iterate x_k we use the shorthand notation $r_k = r(x_k)$, $c_k = c(x_k)$, $J_k = J(x_k)$, etc., and similarly at outer iterates. To avoid interfering with the previous notation for components, we add parentheses to make the distinction clear. For instance, $(x_k)_i$ denotes the i-th component of vector x_k. For any sequence $(u_k)_k$, the limit of a converging subsequence indexed by $\mathcal{K} \subset \mathbb{N}$ will be written $\lim_{k \in \mathcal{K}} u_k$. The orthogonal complement of a subspace $V \subset \mathbb{R}^n$, i.e., the set $\{w \mid \langle w, v \rangle = 0 \text{ for all } v \in V\}$, will be written $V^\perp$.

The rest of the paper is organized as follows. In Section 2, we present our algorithmic framework and discuss implementation details. We study global convergence in Section 3. Numerical experiments are shown in Section 4 and we conclude with perspectives.

2 The algorithm

2.1 Augmented Lagrangian framework

We formally state the first assumptions about problem (1).

Assumption 1 *Residuals r and constraints c are twice continuously differentiable on $\mathbb{R}^n$.*

Assumption 2 *The equality constraint matrix A is full row rank.*

Assumption 3 *The feasible region for the linear constraints Ω is non-empty.*

Our method is based on the augmented Lagrangian (AL) function defined as

$$\Phi(x, \lambda, \mu) := \frac{1}{2}\|r(x)\|^2 + \langle \lambda, c(x) \rangle + \frac{\mu}{2}\|c(x)\|^2, \tag{3}$$

where $\mu > 0$ is a penalty parameter.

We maintain the linear constraints in the formulation of the problem and only penalize the violation of the nonlinear constraints. The gradient is given by

$$\nabla_x \Phi(x, \lambda, \mu) = J(x)^\top r(x) + C(x)^\top \bar{\lambda}(x, \lambda, \mu),$$

where

$$\bar{\lambda}(x, \lambda, \mu) := \lambda + \mu c(x), \tag{4}$$

are the first-order estimates of the Lagrange multipliers. The AL and Lagrangian gradients are related by the identity

$$\nabla_x \Phi(x, \lambda, \mu) = \nabla_x \mathcal{L}(x, \bar{\lambda}(x, \lambda, \mu)). \tag{5}$$

The Hessian is given by

$$\nabla^2_{xx} \Phi(x, \lambda, \mu) = J(x)^\top J(x) + \mu C(x)^\top C(x) + S(x, \lambda, \mu)$$

with the second-order terms explicitly given by

$$S(x, \lambda, \mu) = \sum_{i=1}^{n_r} r_i(x) \nabla^2 r_i(x) + \sum_{i=1}^{n_c} \nabla^2 c_i(x) \bar{\lambda}_i(x, \lambda, \mu). \tag{6}$$

For fixed λ and μ, reformulating problem (1) with function (3) gives the linearly constrained problem

$$\begin{aligned} \min_x \quad & \Phi(x, \lambda, \mu) \\ \text{s.t.} \quad & x \in \Omega. \end{aligned} \tag{7}$$

Moving the nonlinear constraints into the objective simplifies the problem since only linear constraints are left, which enables one to use iterative methods for linearly constrained optimization [40, 47] while still improving feasibility of the nonlinear constraints. Since Ω is a convex set, a first-order critical point of (7) can also be characterized by the condition

$$P_\Omega \left[x_* - \nabla_x \Phi(x_*, \lambda, \mu) \right] = x_*.$$

In our method, as in standard AL methods [35, 38] and more precisely as in the framework of [37] for linearly constrained subproblems, each iterate x_K is an approximate solution of a subproblem of the form (7) satisfying

$$\left\| P_{\mathcal{T}_{\varepsilon_K}(x_K)} \left[-\nabla_x \Phi(x_K, \lambda_K, \mu_K) \right] \right\| \leq \omega_K, \qquad \varepsilon_K = \min(\varepsilon_{\max}, \omega_K), \tag{8}$$

for a fixed vector of multipliers λ_K, penalty parameter μ_K and tolerance ω_K, and a constant $\varepsilon_{\max} > 0$. Here $\mathcal{T}_\varepsilon(x)$ is the tangent cone at x of the linear equalities and of the bounds that lie within distance ε of x, called dominant in [37], which we define precisely in (14) below. For $\varepsilon = 0$, $\mathcal{T}_0(x)$ is the tangent cone of Ω at x, and (8) implies $\| P_\Omega \left[x_K - \nabla_x \Phi(x_K, \lambda_K, \mu_K) \right] - x_K \| \leq \omega_K$ (Lemma 10). Taking $\varepsilon_K > 0$, with $\varepsilon_K \to 0$

as $\omega_K \to 0$, is what allows the inner iterations to terminate (Section 3.1). If the iterate x_K improves the feasibility violation of the nonlinear constraints, a dual ascent step is taken on the Lagrange multipliers by setting

$$\lambda_{K+1} = \lambda_K + \mu_K c_K = \bar{\lambda}(x_K, \lambda_K, \mu_K).$$

If, on the contrary, feasibility has not been improved, the minimization (7) is restarted using an increased penalty parameter $\mu_{K+1} > \mu_K$ to enforce feasibility. The method is outlined in Algorithm 1. Following our notation for iteration-dependent quantities, we respectively write $\nabla_x \Phi(x_K, \lambda_K, \mu_K)$ and $\bar{\lambda}(x_K, \lambda_K, \mu_K)$ in the compact form $\nabla_x \Phi_K$ and $\bar{\lambda}_K$. One first notices that Algorithm 1 has an outer-inner iteration structure, in

Algorithm 1 Augmented Lagrangian algorithm

Require: Initial point $x_0^s \in \Omega$, multiplier estimate λ_0, penalty parameter $\mu_0 > 1$, increase factor $\tau > 1$, tolerance update constants $\omega, \eta, \kappa_\omega, \kappa_\eta, \beta_\omega, \beta_\eta > 0$, stopping tolerances $\omega_*, \eta_* > 0$, threshold $\varepsilon_{\max} > 0$

1: Set initial tolerances $\omega_0 \leftarrow \omega \mu_0^{-\kappa_\omega}$ and $\eta_0 \leftarrow \eta \mu_0^{-\kappa_\eta}$
2: **for** $K = 0, 1, 2, \ldots$ **do**
3: Starting from x_K^s, approximately solve $\min_{x \in \Omega} \Phi(x, \lambda_K, \mu_K)$ to find $x_K \in \Omega$ satisfying (8).
4: **if** $\|c(x_K)\| \leq \eta_K$ **then**
5: **if** $\|P_{\mathcal{T}_{\varepsilon_*}(x_K)}[-\nabla_x \Phi_K]\| \leq \omega_*$, with $\varepsilon_* = \min(\varepsilon_{\max}, \omega_*)$, and $\|c(x_K)\| \leq \eta_*$ **then**
6: **stop** and **return** approximate solution (x_K, λ_K)
7: **end if**
8: Update the Lagrange multipliers $\lambda_{K+1} \leftarrow \bar{\lambda}_K$
9: Set the next starting point $x_{K+1}^s \leftarrow x_K$
10: Leave the penalty parameter unchanged $\mu_{K+1} \leftarrow \mu_K$
11: Decrease the tolerances $\omega_{K+1} \leftarrow \omega_K \mu_{K+1}^{-\beta_\omega}$ and $\eta_{K+1} \leftarrow \eta_K \mu_{K+1}^{-\beta_\eta}$
12: **else**
13: Leave the iterate unchanged $(x_{K+1}^s, \lambda_{K+1}) \leftarrow (x_K^s, \lambda_K)$
14: Increase the penalty parameter $\mu_{K+1} \leftarrow \tau \mu_K$
15: Update tolerances $\omega_{K+1} \leftarrow \omega \mu_{K+1}^{-\kappa_\omega}$ and $\eta_{K+1} \leftarrow \eta \mu_{K+1}^{-\kappa_\eta}$
16: **end if**
17: **end for**

the sense that each iteration requires approximately solving an optimization problem (line 3), which also involves an iterative process. The techniques used for the latter are detailed in the following sections.

Tolerances in Algorithm 1 are updated in such a way that both sequences ω_K and η_K tend to 0 when $K \to \infty$. The update scheme that we outline follows the rules described in [38, Algorithm 14.4.2]. The parameter values used in our implementation are given in Section 2.5.

The projection onto the cone $\mathcal{T}_\varepsilon(x)$ in (8), like the projection onto Ω, is not available in closed form, since it amounts to solving a quadratic program. For our implementation, we use instead a measure based on the reduced gradient and on least-squares estimates of the bound multipliers, as suggested in [37, Section 7.2], which we define now.

For $x \in \Omega$, $\mathcal{A}^+(x)$ and $\mathcal{A}^-(x)$ respectively denote the indices of upper and lower bounds satisfied as equalities (active) at x, and we set $\mathcal{A}(x) := \mathcal{A}^+(x) \cup \mathcal{A}^-(x)$, so that

$$i \in \mathcal{A}(x) \iff x_i \in \{l_i, u_i\}.$$

We also introduce the notion of tangent space at a point x, written $T(x)$ and defined as the set of directions d such that the same constraints are satisfied with equality at both x and $x + d$. In our case, this corresponds to the linear equality constraints and the active bounds. Formally,

$$T(x) := \{d \in \mathbb{R}^n \mid Ad = 0,\ d_i = 0 \text{ for all } i \in \mathcal{A}(x)\}\,. \tag{9}$$

At a given point (x, λ) and parameter μ, the reduced gradient is defined as

$$P_{T(x)}\left[\nabla_x \Phi(x, \lambda, \mu)\right],$$

where $P_{T(x)}[\cdot]$ denotes the orthogonal projection operator onto the tangent space $T(x)$. Provided that the multipliers associated with the active bounds are of appropriate sign, the quantity $\|P_{T(x)}\left[\nabla_x \Phi(x, \lambda, \mu)\right]\|$ is an appropriate criticality measure used in optimization algorithms for linearly constrained optimization such as MINOS [48] and LSNNO [49]. Projections on the tangent space also have the advantage of being available in closed form. The subspace $T(x)$ is the null space of the matrix

$$\tilde{A} = \begin{pmatrix} A \\ E_{\mathcal{A}}^\top \end{pmatrix}, \tag{10}$$

where $E_{\mathcal{A}}$ is formed by the columns $(e_i)_{i \in \mathcal{A}(x)}$ of the identity. We assume throughout that these active constraints remain non-degenerate.

Assumption 4 *For every $x \in \Omega$, the matrix (10) formed from the bounds active at x has full row rank.*

Assumption 4 strengthens Assumption 2: it is a linear independence (non-degeneracy) condition on the active linear constraints, which holds, in particular, when no active bound is linearly dependent on the rows of A. Under this assumption, projecting on $T(x)$ is equivalent to applying the orthogonal projection operator

$$\tilde{P} = I - \tilde{A}^\top \left(\tilde{A}\tilde{A}^\top\right)^{-1} \tilde{A}. \tag{11}$$

We will provide more details on how the projections are calculated in practice in Section 2.5. Under Assumption 4, every vector $v \in \mathbb{R}^n$ admits the unique decomposition

$$v = P_{T(x)}\,[v] + A^\top \xi + E_{\mathcal{A}}\theta, \tag{12}$$

where $(\xi, \theta) \in \mathbb{R}^m \times \mathbb{R}^{|\mathcal{A}(x)|}$ solves the least-squares problem $\min \|v - \tilde{A}^\top(\xi,\theta)\|$. For $v = \nabla_x \Phi(x,\lambda,\mu)$, the components θ_i, $i \in \mathcal{A}(x)$, are least-squares estimates of the multipliers of the active bounds. They come at no extra cost, since (ξ,θ) is the auxiliary vector y of the system (36) solved to compute $\tilde{P}v$. At a first-order critical point, the multiplier of an active lower bound is nonnegative and that of an active upper bound is nonpositive (see (47) below). We measure the violation of these sign conditions by the vector θ^{w} with components

$$\theta_i^{\mathrm{w}} = \begin{cases} \max(0, -\theta_i) & \text{if } i \in \mathcal{A}^-(x) \setminus \mathcal{A}^+(x), \\ \max(0, \theta_i) & \text{if } i \in \mathcal{A}^+(x) \setminus \mathcal{A}^-(x), \\ 0 & \text{if } i \in \mathcal{A}^+(x) \cap \mathcal{A}^-(x), \end{cases}$$

the last case corresponding to fixed variables, whose multiplier may have any sign, and we define the criticality measure

$$\pi(x,\lambda,\mu) := \max\left(\left\|P_{T(x)}\,[\nabla_x \Phi(x,\lambda,\mu)]\right\|_\infty, \|\theta^{\mathrm{w}}\|_\infty\right). \tag{13}$$

The reduced gradient alone vanishes at every point where $\nabla_x\Phi$ lies in the row space of $\tilde{A}$, whatever the signs of the bound multipliers, in particular at every vertex of Ω. It cannot certify criticality on its own, and an algorithm stopping on it may terminate at a point that is not critical.

More generally, for $\varepsilon \geq 0$, we call ε-active the bounds of

$$\mathcal{A}_\varepsilon^-(x) = \{i \mid x_i - l_i \leq \varepsilon\}, \qquad \mathcal{A}_\varepsilon^+(x) = \{i \mid u_i - x_i \leq \varepsilon\}, \qquad \mathcal{A}_\varepsilon(x) = \mathcal{A}_\varepsilon^-(x) \cup \mathcal{A}_\varepsilon^+(x),$$

and we define $T_\varepsilon(x)$, $\tilde{A}_\varepsilon$, the decomposition (12) and the vector θ^{w} as above, with $\mathcal{A}_\varepsilon(x)$ in place of $\mathcal{A}(x)$. For a vector $v \in \mathbb{R}^n$, we denote by $\pi_\varepsilon(x,v)$ the right-hand side of (13) computed in this way for v in place of $\nabla_x\Phi$, and we write $\pi_\varepsilon(x,\lambda,\mu) = \pi_\varepsilon(x, \nabla_x\Phi(x,\lambda,\mu))$, so that $\pi_0 = \pi$. For ε below a threshold $\bar{\varepsilon} > 0$ that only depends on Ω, $\tilde{A}_\varepsilon$ has full row rank and the decomposition is unique (Lemma 1). Finally, we denote by

$$\mathcal{T}_\varepsilon(x) = \left\{ d \in \mathbb{R}^n \;\middle|\; \begin{array}{l} Ad = 0,\ d_i \geq 0 \text{ for } i \in \mathcal{A}_\varepsilon^-(x) \setminus \mathcal{A}_\varepsilon^+(x), \\ d_i \leq 0 \text{ for } i \in \mathcal{A}_\varepsilon^+(x) \setminus \mathcal{A}_\varepsilon^-(x),\ d_i = 0 \text{ for } i \in \mathcal{A}_\varepsilon^-(x) \cap \mathcal{A}_\varepsilon^+(x) \end{array} \right\} \tag{14}$$

the tangent cone of the linear equalities and of the ε-active bounds, and by $\mathcal{N}_\varepsilon(x)$ its polar cone, generated by the rows of A and their opposites, the vectors $-e_i$, $i \in \mathcal{A}_\varepsilon^-(x)$, and the vectors e_i, $i \in \mathcal{A}_\varepsilon^+(x)$. For $\varepsilon = 0$, they are the tangent and normal cones of Ω at x, and, by the Moreau decomposition, $\|P_{\mathcal{T}_\varepsilon(x)}\,[-v]\| = \operatorname{dist}(-v, \mathcal{N}_\varepsilon(x))$ for all

$v \in \mathbb{R}^n$. Our implementation stops the inner iterations of outer iteration K when

$$\pi_{\varepsilon_K}(x_K, \lambda_K, \mu_K) \leq \omega_K, \tag{15}$$

with ε_K as in (8). Lemma 10, a reformulation of [37, Lemma 7.1 and (7.3)], shows that $\|P_{\mathcal{T}_\varepsilon(x)}[-\nabla_x \Phi(x, \lambda, \mu)]\| \leq \sqrt{n}\,\pi_\varepsilon(x, \lambda, \mu)$, so that (15) implies (8) with ω_K replaced by $\sqrt{n}\,\omega_K$, and that π vanishes exactly at the first-order critical points of (7). The role of the threshold ε_K, which keeps the Cauchy step from being arbitrarily short near the bounds, is explained in Section 3.1. Likewise, the final test of Algorithm 1 is implemented as $\pi_{\varepsilon_*}(x_K, \lambda_K, \mu_K) \leq \omega_*$ with $\varepsilon_* = \min(\varepsilon_{\max}, \omega_*)$.

2.2 Inner minimization process

In this section, we describe the inner minimization process used to produce the outer iterates of Algorithm 1. Since the current multiplier estimates λ and the penalty parameter μ are fixed, the objective function for the inner minimization only depends on x so we use the shorthand notation $\varphi : x \mapsto \Phi(x, \lambda, \mu)$. Given a point $x_0 \in \Omega$, multiplier estimates λ and a penalty parameter μ, we aim to find an approximate solution of the problem

$$\begin{aligned} \min_x \quad & \varphi(x) \\ \text{s.t.} \quad & x \in \Omega, \end{aligned} \tag{16}$$

and such that

$$\|P_{T(x)}[\nabla \varphi(x)]\| \leq \omega,$$

for a tolerance $\omega > 0$. This task also involves an iterative process, for which the iterates will be indexed with subscript k but are distinct from the outer iterates of Algorithm 1. At each iteration k, we compute a step s_k and recursively update the iterate by $x_{k+1} = x_k + s_k$. The step is obtained after minimizing a model of the objective function φ around x_k. We consider the quadratic model

$$m_k(x) = \varphi_k + \langle g_k, x - x_k \rangle + \frac{1}{2} \langle x - x_k, H_k(x - x_k) \rangle, \tag{17}$$

where $\varphi_k := \varphi(x_k)$, $g_k := \nabla \varphi_k$ and H_k is a symmetric approximation of the true Hessian $\nabla^2_{xx} \varphi_k$. Using the latter would negatively impact the performance of the algorithm since computing the second-order terms (6) requires too much time and storage in practice. We discuss the aspects of the methods relative to the choice of approximation in Section 2.4. Model (17) is appropriate to describe the algorithm in terms of the iterate. When discussing subproblems, we prefer to emphasize the step and would then use the following model of the primal reduction $\varphi(x_k + s) - \varphi_k$, given by

$$q_k(s) = \frac{1}{2} \langle s, H_k s \rangle + \langle g_k, s \rangle.$$

The two formulations are related by $q_k(x - x_k) = m_k(x) - m_k(x_k)$.

We seek to compute the step s_k as an approximate solution of the program

$$\min_s \quad q_k(s)$$

$$\begin{aligned} \text{s.t.} \quad & x_k + s \in \Omega \\ & \|s\|_\infty \leq \Delta_k. \end{aligned}$$

Here, s denotes the unknown of the subproblem, whose solution s_k is the step used to compute the new iterate $x_{k+1} = x_k + s_k$.

Since x_0 is feasible, the constraints $x_k + s \in \Omega$ are satisfied at every iteration as long as the steps satisfy

$$As = 0, \qquad l - x_k \leq s \leq u - x_k,$$

for all k.

In our method, we also incorporate a trust-region strategy to control and assess the quality of a step. Therefore, we add to each subproblem the constraint $\|s\|_\infty \leq \Delta_k$ with a radius $\Delta_k > 0$, which reflects the domain on which we believe the model approximates the true function well. We use the ℓ_∞ norm because the intersection of the resulting trust region with Ω is again a polyhedron of the form (2): the radius constraint only tightens the bounds and leaves the linear equalities untouched, so the subproblem retains the structure of a problem with bounds and linear equalities that our projection machinery handles directly. Since $\|s\|_\infty \leq \Delta_k$ amounts to imposing $s_i \in [-\Delta_k, \Delta_k]$ for all i, we can rewrite this subproblem

$$\min_s \quad q_k(s) \tag{18a}$$

$$\text{s.t.} \quad As = 0 \tag{18b}$$

$$l_k \leq s \leq u_k, \tag{18c}$$

where l_k and u_k are iteration-dependent bounds having components $(l_k)_i = \max(-\Delta_k, l_i - (x_k)_i)$ and $(u_k)_i = \min(\Delta_k, u_i - (x_k)_i)$ for all $i = 1, \ldots, n$.

The success of an iteration and the update of the trust region are evaluated by the ratio

$$\rho_k = \frac{\varphi(x_k + s_k) - \varphi_k}{q_k(s_k)}. \tag{19}$$

We follow the standard step acceptance criteria [38], which require constants $\eta_1, \eta_2, \gamma_1, \gamma_2$ such that

$$0 < \eta_1 \leq \eta_2 < 1 \text{ and } 0 < \gamma_1 \leq \gamma_2 < 1. \tag{20}$$

If $\rho_k \geq \eta_1$, the step is accepted and the trust region is expanded. Otherwise, the step is rejected and the region reduced. A typical scheme to update the radius Δ_k would be to set

$$\Delta_{k+1} \in \begin{cases} [\Delta_k, \infty) & \text{if } \rho_k \geq \eta_2 \\ [\gamma_2 \Delta_k, \Delta_k] & \text{if } \rho_k \in [\eta_1, \eta_2) \\ [\gamma_1 \Delta_k, \gamma_2 \Delta_k] & \text{if } \rho_k < \eta_1 \end{cases} \tag{21}$$

The procedure employed to solve subproblem (18) is described in Algorithm 2 and the latter is analyzed in the rest of this section.

Algorithm 2 Trust region inner iteration algorithm

Require: Initial point $x_0 \in \Omega$, radius $\Delta_0 > 0$, constants $\eta_1, \eta_2, \gamma_1, \gamma_2$ satisfying conditions (20).

Set $k \leftarrow 0$
repeat
  Compute the model m_k
  Compute a step s_k that sufficiently reduces the model
  Compute the ratio ρ_k (19)
  if $\rho_k \geq \eta_1$ **then** set $x_{k+1} \leftarrow x_k + s_k$ and update H_{k+1}
  else set $x_{k+1} \leftarrow x_k$ and $H_{k+1} \leftarrow H_k$
  end if
  Set Δ_{k+1} according to (21)
  Increment $k \leftarrow k + 1$
until $\pi_\varepsilon(x_k) \leq \omega$

2.3 Step computation

We consider the current feasible iterate x_k and its associated approximate quadratic model m_k (17). The step is computed in two phases.

Cauchy step

We first compute a Cauchy step s_k^C that ensures a decrease of the objective function sufficient to establish global convergence of the inner minimization algorithm. Next, we further minimize the objective function by exploring the subspace defined by the constraints active at the Cauchy point $x_k^C := x_k + s_k^C$. This approach is commonly used in gradient projection techniques [2, Section 16.7] and there are different strategies to compute a Cauchy point [40, 45, 46]. Because of the structure of the linear constraints, we cannot directly project on the whole set Ω in practice, but we can exploit prior knowledge of the active bounds to compute the projection iteratively. That is the reason why we compute our Cauchy step by finding the first local minimizer of the model along a piecewise linear path that follows the projected steepest descent direction on nested faces of the feasible set, a bound being kept fixed once it is reached. The path starts from a working set selected among the ε-active bounds by the signs of their multiplier estimates (Section 3.1 and Appendix A). When Ω is a box, this path is the projected gradient path

$$s_k(t) = P_\Omega\left[x_k - t g_k\right] - x_k \text{ for } t \geq 0. \tag{22}$$

Over a general polyhedron, the active sets along (22) need not be nested, and the two paths may differ. Our procedure – described in Appendix A – is an adaptation to the polyhedral case of algorithm SBMIN [40] used for the inner minimization phase of the LANCELOT solver [39].

Subspace minimization

In order to improve the rate of convergence of the algorithm, we want the total step to achieve a better reduction than the Cauchy step. To do so, we build the next iterate x_{k+1} after a finite sequence of M minor iterates $x_{k,1}, \dots, x_{k,M+1}$. The sequence starts at the Cauchy point and ends at the next iterate, i.e., $x_k + s_k^C = x_{k,1}$ and $x_{k+1} = x_{k,M+1}$. This type of approach has been shown to be effective for general optimization with bound constraints [46] and has also been incorporated into other AL algorithms for the inner loop minimization [39, 43].

Each minor iterate is defined after the previous one and is decomposed into

$$x_{k,j+1} = x_{k,j} + w_{k,j},$$

where $w_{k,j}$ is a descent direction for the quadratic model q_k. For each minor iterate, we require

$$x_{k,j} \in \Omega, \quad \|x_{k,j} - x_k\|_\infty \leq \Delta_k, \quad \mathcal{A}(x_k^C) \subseteq \mathcal{A}(x_{k,j}). \tag{23}$$

The first two conditions are merely that each minor iterate is feasible and the associated step lies within the trust region, while the last condition means that we can only add active bounds during this process. Note that in this context, a bound can become active with respect to the trust region and not only the original bounds on the variables. Likewise, the variables fixed at their current values at the start of the Cauchy step, because of an ε-active bound, remain fixed during the minor iterations. We also require the sufficient decrease between two successive minor iterates

$$m_k(x_{k,j+1}) \leq m_k(x_{k,j}), \qquad j = 1, \dots, M. \tag{24}$$

After each minor iteration, the corresponding step is $s_{k,j} := x_{k,j} - x_k$.

The search direction $w_{k,j}$ is an approximate minimizer of the subproblem

$$\begin{aligned} \min_w \quad & m_k(x_{k,j} + w) && \text{(25a)} \\ \text{s.t.} \quad & Aw = 0 && \text{(25b)} \\ & w_i = 0, \qquad i \in \mathcal{A}(x_{k,j}). && \text{(25c)} \end{aligned}$$

At a minor iteration, the free variables, indexed by $\mathcal{F}(x_{k,j})$, are implicitly subject to the bounds

$$\left(l^{(k,j)}\right)_i \leq w_i \leq \left(u^{(k,j)}\right)_i, \qquad i \in \mathcal{F}(x_{k,j}) \tag{26}$$

where $l^{(k,j)} := l_k - s_{k,j}$ and $u^{(k,j)} := u_k - s_{k,j}$.

The minor subproblem (25) is solved by applying the projected conjugate gradient [47] method with the previous minor iterate $x_{k,j}$ as a starting point. Three termination cases can occur. First, a direction can be generated such that one of the bounds (26) is violated for one of the free variables. When this happens, the search direction is scaled so that the associated component lies at the bound and the CG iterations stop. The second case occurs when we generate a direction of negative curvature and is handled similarly to the first one, i.e., we modify the direction so that all the variables remain within their bounds. The third case is the normal termination

one and occurs when a local minimizer – with respect to a given tolerance – has been found. In all cases, we return the obtained search direction $w_{k,j}$ and perform the projected line search to compute the point $x_{k,j+1}$ such that eqs. (23) and (24) are verified. The handling of the termination cases differs in the continuation of the procedure. The last two cases (negative curvature and local optimality) terminate the minor iteration loop. On the contrary, if the CG iterations stopped because a bound was hit, we go back to solving (25) using the projected conjugate gradient method with $x_{k,j+1}$ as a starting point and $\mathcal{A}(x_{k,j+1}) \supset \mathcal{A}(x_{k,j})$ as a new set of fixed components. The specification of the projected conjugate gradient method applied to solving (25) is outlined in Algorithm 3. The operations occurring at lines 1 and 16 are projections onto the

Algorithm 3 The projected conjugate gradient method applied to (25)

1: Set $w \leftarrow 0,\ r \leftarrow H_k(x_{k,j} - x_k) + g_k,\ v \leftarrow \tilde{P}r,\ p \leftarrow -v$
2: Set tolerance $\varepsilon_{\text{cg}} \leftarrow \kappa_{\text{cg}}\|v\|$
3: **repeat**
4: **if** $\langle p, H_k p\rangle \leq 0$ **then**
5: Set $w^+ \leftarrow w + \gamma p$ where γ is the smallest factor such that $w_i^+ \in \{l_i^{(k,j)}, u_i^{(k,j)}\}$ for $i \in \mathcal{F}(x_{k,j})$
6: **STOP**
7: **end if**
8: Set $\alpha \leftarrow \langle r, v\rangle \,/\, \langle p, H_k p\rangle$
9: Set $w^+ \leftarrow w + \alpha p$
10: **if** w^+ violates a bound **then**
11: $w^+ \leftarrow w + \gamma p$ where γ is the smallest factor such that $w_i^+ \in \{l_i^{(k,j)}, u_i^{(k,j)}\}$
12: for $i \in \mathcal{F}(x_{k,j})$
13: **STOP**
14: **end if**
15: Set $r^+ \leftarrow r + \alpha H_k p$
16: Set $v^+ \leftarrow \tilde{P}r^+$
17: **if** $\sqrt{\langle r^+, v^+\rangle} < \varepsilon_{\text{cg}}$ **then STOP**
18: **end if**
19: Set $\beta \leftarrow \langle r^+, v^+\rangle \,/\, \langle r, v\rangle$
20: Set $p \leftarrow -v^+ + \beta p$
21: Set $w \leftarrow w^+,\ r \leftarrow r^+,\ v \leftarrow v^+$
22: **until** $2(n - m - |\mathcal{A}(x_{k,j})|)$ iterations have been done
23: **return** w^+

null space defined by the constraints (25b)–(25c).

An interesting feature would be to complement each search direction by a steplength computed by a projected search [45, 46]. This allows one to add more than one constraint at each minor iteration to the active set. The downside is that it would also require computing the projection of the gradient direction (22) on the feasible set Ω at every new trial value of the steplength. When Ω only contains bound constraints, the projection is trivial and cheap to compute. The linear equality case is more costly

but the projections can also be computed directly. When Ω is of the form (2), the projection is less trivial, as it amounts to solving a minimum-distance quadratic program over Ω at every trial steplength. Performing such a projection repeatedly would require a dedicated quadratic programming solver – an external dependency whose per-trial cost would outweigh the benefit of enlarging the active set more aggressively – which we prefer to avoid.

We stop the minor-iterations procedure when the reduced gradient associated with the current active set is small enough. More formally, assume we performed the $j+1$ minor minimizations and let $T_{k,j}$ be the tangent space at $x_{k,j}$ with respect to the active bounds in $\mathcal{A}(x_{k,j})$. We stop the minor iteration loop whenever

$$\left\|P_{T_{k,j}}\left[\nabla m_k(x_{k,j+1})\right]\right\| \leq \kappa_{\text{mlt}} \left\|P_{T_{k,j}}\left[\nabla m_k(x_k)\right]\right\|,$$

with $\kappa_{\text{mlt}} \in (0,1)$. This inequality estimates whether relative progress can still be made in the tangent subspace spanned by the free variables.

2.4 Structured Hessian approximation

We now discuss how the Hessian of the AL is iteratively updated when we define the quadratic model of iteration k in Algorithm 2. To set up the context and notation of this paragraph, we wish to approximate the true Hessian

$$\nabla^2\varphi(x_k) = J_k^\top J_k + \mu C_k^\top C_k + S_k,$$

by a symmetric matrix H_k. We recall that the need for an approximation mainly comes from the fact that evaluating the second-order terms in S_k requires too much time and storage to be done in practice, especially for problems with a large number of variables and residuals.

The first approximation we can think of, and the simplest one, is obtained after merely linearizing the residuals and constraints in expression (3), which gives

$$H_k = J_k^\top J_k + \mu C_k^\top C_k. \tag{27}$$

The GN approximation (27) – in reference to its unconstrained counterpart – is computationally cheap as it involves first-order derivative quantities already required to evaluate the gradient. This matrix is always positive semidefinite. It is positive definite – which guarantees the convexity of the quadratic subproblems (18) – if and only if the stacked matrix $\begin{pmatrix} J_k \\ \sqrt{\mu} C_k \end{pmatrix}$ has full column rank, that is, $\ker J_k \cap \ker C_k = \{0\}$. However, it is known to exhibit a slower rate of convergence on problems with nonzero residuals at the solution. This downside is amplified when we are to approximate the Hessian of the Lagrangian, or the AL in our case. Indeed, setting S_k to the zero matrix not only neglects the contribution of the residuals to the curvature of the model, but also neglects the contribution of the Lagrange multipliers, for which there is no a priori reason to vanish.

We thus need to take into account the full Hessian to form an approximation. Looking at the literature on this subject, one can observe that there is a variety of

approaches focusing on the unconstrained case [8, 13, 15–18], as it is a major challenge in improving the efficiency of algorithms for problems with nonzero residuals at the solution. Nevertheless, relevant parallels can be drawn with the AL situation, or the constrained case in general, since these studies explore techniques to deal with second-order terms. Because the first-order terms are readily available and provide curvature information, only the second-order components need to be approximated. Therefore, we look for an approximation of the form

$$H_k = B_k^{\text{GN}} + B_k, \tag{28}$$

where B_k^{GN} is the right-hand side of (27) and B_k is a symmetric approximation of S_k.

Structured approximations of the AL Hessian have been studied for general objective functions [30] but our approach is closer to the one employed in [26]. In the latter, the authors base their approximation on a secant equation derived from a heuristic initially introduced in the unconstrained case [13] that they adapted to approximate the Hessian of the Lagrangian and use it in an SQP algorithm for constrained nonlinear least-squares. This strategy is also at the heart of the SQN method from the popular package for unconstrained nonlinear least-squares NL2SOL [8, 9]. The idea is the following. If we were to approximate each matrix term of the sum (6), we would have

$$B_{k+1} = \sum_{i=1}^{n_r} r_i(x_{k+1}) B_{k+1}^{r_i} + \sum_{i=1}^{n_c} \bar{\lambda}_i(x_{k+1}, \lambda, \mu) B_{k+1}^{c_i}, \tag{29}$$

with $B_{k+1}^{r_i} \approx \nabla^2 r_i(x_{k+1})$ and $B_{k+1}^{c_i} \approx \nabla^2 c_i(x_{k+1})$. To get an accurate approximation, given a step s_k, it is reasonable to require

$$B_{k+1}^{r_i} s_k = \nabla r_i(x_{k+1}) - \nabla r_i(x_k) \qquad \text{and} \qquad B_{k+1}^{c_i} s_k = \nabla c_i(x_{k+1}) - \nabla c_i(x_k), \tag{30}$$

i.e., that each Hessian maps the change in the variables to the change in the gradients. Noticing that, for each i, the vectors $\nabla r_i(x_{k+1}) - \nabla r_i(x_k)$ and $\nabla c_i(x_{k+1}) - \nabla c_i(x_k)$ are respectively the i-th rows of $(J_{k+1} - J_k)^\top$ and $(C_{k+1} - C_k)^\top$, summing over the residuals and constraints indices all the terms of (30) gives, by (29), the structured secant equation

$$B_{k+1} s_k = (J_{k+1} - J_k)^\top r_{k+1} + (C_{k+1} - C_k)^\top \bar{\lambda}_{k+1}. \tag{31}$$

To follow the conventional notation of quasi-Newton methods, we will denote the right-hand side of (31) by $\tilde{y}_k$ to insist on its distinction with the standard term $y_k := g_{k+1} - g_k$. Note that, as in SQN methods for unconstrained least-squares [18], since λ and μ are fixed during the inner minimization,

$$y_k - \tilde{y}_k = J_k^\top (r_{k+1} - r_k) + \mu C_k^\top (c_{k+1} - c_k) = B_k^{\text{GN}} s_k + O\left(\|s_k\|^2\right).$$

As $B_{k+1}^{\text{GN}} s_k = B_k^{\text{GN}} s_k + O(\|s_k\|^2)$ as well, the resulting approximation $H_{k+1} = B_{k+1}^{\text{GN}} + B_{k+1}$ satisfies the standard secant equation $H_{k+1} s_k = y_k$ up to second-order terms in the step, and exactly if the residuals and the constraints are affine.

One could instead enforce the standard secant equation exactly, by replacing $\tilde{y}_k$ with $y_k - B_{k+1}^{\mathrm{GN}} s_k$ in the update of B_k. This is the default choice of [57] for structured secant updates, which, in the experience of [9], is inferior to choices exploiting the structure of each application, such as the NL2SOL choice that (31) extends. In the AL setting, it has moreover a drawback without counterpart in the unconstrained case. If the second derivatives of r and c are Lipschitz continuous, expanding r and c around x_k gives

$$\begin{aligned} y_k - B_{k+1}^{\mathrm{GN}} s_k - \tilde{y}_k = & -\sum_{i=1}^{n_r} (J_k s_k)_i \, \nabla^2 r_i(x_k) s_k - \tfrac{1}{2} J_k^T q_k^r + O\big((1+\mu)\|s_k\|^3\big) \\ & - \mu \Big(\sum_{i=1}^{n_c} (C_k s_k)_i \, \nabla^2 c_i(x_k) s_k + \tfrac{1}{2} C_k^T q_k^c \Big), \end{aligned} \tag{32}$$

where $(q_k^r)_i = \langle s_k, \nabla^2 r_i(x_k) s_k \rangle$ and $(q_k^c)_i = \langle s_k, \nabla^2 c_i(x_k) s_k \rangle$. The right-hand side is quadratic in s_k, so that no fixed matrix maps s_k to it, and its second line is weighted by the penalty parameter itself. By contrast, $\tilde{y}_k$ depends on μ only through the shifted multipliers $\bar{\lambda}_{k+1}$, and $\tilde{y}_k - S_{k+1} s_k$ is of order $(\|r_{k+1}\| + \|\bar{\lambda}_{k+1}\|)\|s_k\|^2$, where the shifted multipliers $\bar{\lambda}_K$ at the outer iterates converge to λ_* by (49), whatever the behaviour of μ_K. Enforcing $H_{k+1} s_k = y_k$ exactly would thus feed into B_{k+1} a linearization error of the GN term proportional to μ, which matters when the penalty parameter has to be increased to reach feasibility. In the unconstrained case, $\mu = 0$ and only the first line of (32) remains, a term of order $\|s_k\|^2$ without any large factor. [Addition to be supported by numerical tests.]

The closest existing proposal from the approximation (31) is the one in [26]. However, the two works differ in the algorithmic framework within which the approximation is used. In [26], the authors operate within an SQP method, so their approximation targets the Hessian of the Lagrangian and is used to build a quadratic subproblem in which the constraints appear linearized, whereas ours targets the Hessian of the AL and is used within a minimization over a fixed polyhedron. In particular, the curvature contributed by the nonlinear constraints enters our secant equation through the shifted multipliers $\bar{\lambda}_{k+1}$, which are fixed during the inner minimization, and not through Lagrange multiplier estimates updated after every iteration.

We can now derive an update formula from equation (31). In our implementation, we apply the SR1 update

$$B_{k+1} = B_k + \frac{(\tilde{y}_k - B_k s_k)(\tilde{y}_k - B_k s_k)^T}{\langle \tilde{y}_k - B_k s_k, s_k \rangle}, \tag{33}$$

if $\langle \tilde{y}_k - B_k s_k, s_k \rangle \neq 0$. When the denominator in (33) is zero, we simply set $B_{k+1} = B_k$. To avoid numerical errors, we apply the update only when

$$|\langle \tilde{y}_k - B_k s_k, s_k \rangle| > \kappa_{\mathrm{sds}} \, (1 + \|s_k\| \|\tilde{y}_k - B_k s_k\|) \, , \tag{34}$$

for $\kappa_{\text{sds}} \in (0, 1)$. Inequality (34) adds an absolute term to the most common safeguard used in SR1 methods [2, Section 6.2], so that the update is also skipped when $\|s_k\|\|\tilde{y}_k - B_k s_k\|$ is negligible.

The choice of the SR1 update formula is motivated by its robustness in the least-squares context, as the exhaustive benchmarks in [18] show, and its tendency to better approximate the true Hessian with each iteration [58]. The approximation could thus be indefinite, which could be considered as a weakness compared to BFGS or DFP updates, which are guaranteed to be positive definite as long as this is the case for the initial approximation B_0. However, as highlighted in Algorithm 3, the use of the conjugate gradient method in a trust-region framework handles, and even exploits, the potential non-convexity of the subproblems. Moreover, this update does not require a curvature condition $\langle s_k, y_k \rangle > 0$ to be satisfied. In other works on the constrained case, SQN methods are used to approximate the projected Hessian of the Lagrangian [27] – or of a merit function [28] – in the null space of the active constraints. The BFGS update is then preferred because this matrix is, under standard assumptions, positive semidefinite, according to the second-order necessary conditions.

We end our description of the Hessian approximation by discussing hybrid updates, a feature commonly used in SQN methods for nonlinear least-squares, either in the unconstrained [8, 10, 11] or constrained [26, 27] case. The idea is to test at every iteration whether the problem has small or large residuals at the solution. In the latter case, a structured update is used to increase the accuracy of the approximated Hessian whereas in the former, the GN approximation is employed, as it is more likely to correspond to the true Hessian. Note that the update, such as (33), is still computed at every iteration to continue accumulating curvature information. We use a strategy inspired by [11] adapted to the constrained case in [27] that is based on the asymptotic behavior of the ratio

$$\zeta_k = \frac{\varphi(x_k) - \varphi(x_{k+1})}{\varphi(x_k)}.$$

The approximated Hessian is then updated following the switching rule

$$H_{k+1} = \begin{cases} B_{k+1}^{\text{GN}} + B_{k+1} & \text{if } \zeta_k \leq \kappa_{\text{hyb}} \\ B_{k+1}^{\text{GN}} & \text{otherwise,} \end{cases} \tag{35}$$

where κ_{hyb} is a parameter in $(0, 1)$. In [27], the ratio is computed with the augmented Lagrangian used as merit function by their SQP method. As we will see in Section 4, it has an important impact on the performance of the algorithm.

2.5 Summary and implementation details

We end our discussion by summarizing the relations between outer, inner and minor iterates and the default values for the parameters of the algorithm presented in this section. Recall that the outer iterates are indexed with K and the inner iterates with k. The outer iterates x_K are the main iterates of the algorithm and are approximate minimizers of the AL. Each outer iterate is formed after a sequence of inner iterates $(x_k)_k$, linked by $x_{k+1} = x_k + s_k$, where s_k is an approximate solution of the QP (18). In turn, each inner iterate is formed after a sequence of M minor iterates $x_{k,j}$, linked

by $x_{k,j+1} = x_{k,j} + w_{k,j}$, where $w_{k,j}$ is a descent direction obtained by applying the projected conjugate gradient Algorithm 3. The relations between the three levels of iterations are illustrated in Figure 1.

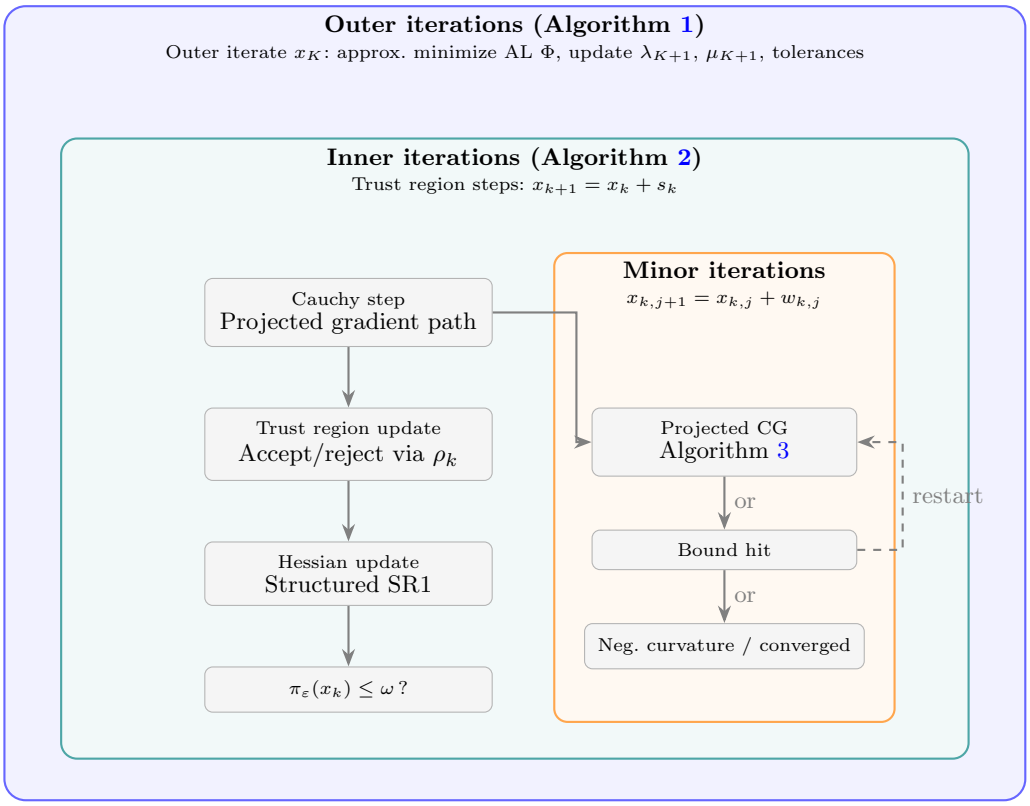


Fig. 1: Hierarchy of the three iteration levels in the AL algorithm. The outer loop updates the Lagrange multipliers and penalty parameter. Each outer iteration triggers a sequence of trust-region inner iterations, and each inner iteration computes its step via a Cauchy point followed by minor iterations using the projected conjugate gradient method.

Computing projections

From our description of the step computation, we can see that whether during the Cauchy step calculation phase or during the minor iteration loop, the active set – and thus the projection operator $\tilde{P}$ at (11) – changes over the course of an iteration. It is therefore worth examining how this operator is updated. One can compute the projection of a vector v, i.e., $\tilde{P}v$, by first solving, for an auxiliary variable y, the linear system

$$\left(\tilde{A}\tilde{A}^\top\right) y = \tilde{A}v, \tag{36}$$

and then retrieving the projection as

$$v - \tilde{A}^\top y. \tag{37}$$

This approach, referred to as the *normal equations* approach [47], requires solving the system (36). This is done by using the Cholesky decomposition of $\tilde{A}\tilde{A}^\top = LL^\top$ – with L lower triangular – and system (36) is thus reduced to two successive lower triangular systems. The latter factorization is well defined since the matrix $\tilde{A}\tilde{A}^\top$ is symmetric by construction and positive definite by Assumption 4. Moreover, the block structure of $\tilde{A}\tilde{A}^\top$ can be exploited to incrementally maintain the L factor – instead of forming it from scratch – across the changes in the active set. Given an active set $\mathcal{A} \subset \{1, \ldots, n\}$ of cardinality $p < n - m$, and writing $A_{\mathcal{A}} := AE_{\mathcal{A}}$ for the restriction of A to the columns indexed by $\mathcal{A}$, $\tilde{A}\tilde{A}^\top$ can be written

$$\begin{bmatrix} AA^\top & A_{\mathcal{A}} \\ A_{\mathcal{A}}^\top & I_p \end{bmatrix}. \tag{38}$$

From (38), one can show [59, see Section 4.2.9] that the L factor has the corresponding block pattern

$$L = \begin{bmatrix} L_{11} & 0 \\ L_{21} & L_{22} \end{bmatrix},$$

where L_{11} is the $m \times m$ Cholesky factor of $AA^\top$, $L_{21} := (L_{11}^{-1} A_{\mathcal{A}})^\top$ is of size $p \times m$, and L_{22} is the $p \times p$ Cholesky factor of $G := I_p - L_{21}L_{21}^\top$. The matrix A being fixed, forming its Cholesky decomposition and $L_{11}^{-1}A$ just has to be done once at the start of the algorithm. Therefore, L_{11} and L_{21} – the latter given by extracting the columns of $L_{11}^{-1}A$ indexed by $\mathcal{A}$ – are both available quantities and only the L_{22} part needs to be maintained. Since adding an index to $\mathcal{A}$ results in adding a column to $A_{\mathcal{A}}$, and thus in appending a row and a column to G, the new factor L_{22} can be updated incrementally by applying a bordered update to the existing one in $\mathcal{O}(p^2)$ flops, which is less costly than the $\mathcal{O}((m+p)^3)$ time complexity required to form the entire L factor from scratch.

Bound constrained problems

Our intention with the work presented in this paper was to design an algorithm able to directly handle linear constraints of the form (2) but we also accept problems where linear constraints are only bounds and then

$$\Omega = \{x \in \mathbb{R}^n \mid l \leq x \leq u\}.$$

The framework we have presented remains valid for this formulation. The only practical difference is in the computation of projections. In the bound-constrained case, the projection of a vector x on the set Ω has components

$$\left(P_\Omega\left[x\right]\right)_i = \begin{cases} l_i & \text{if } x_i < l_i \\ x_i & \text{if } x_i \in [l_i, u_i] \\ u_i & \text{if } x_i > u_i \end{cases} \tag{39}$$

In our implementation, the only practical difference with the polyhedral case is that we can directly use the norm of the projected gradient

$$\|x_K - P_\Omega\left[x_K - \nabla_x \Phi_K\right]\|,$$

as a criticality measure. Note that the projections onto tangent spaces are also simplified, as they are now given by setting the components corresponding to active bounds to 0.

Parameter conditions

Algorithm 1 requires $\mu_0 > 1$, $\tau > 1$, positive tolerance constants $\omega, \eta, \kappa_\omega, \kappa_\eta, \beta_\omega, \beta_\eta$ and positive stopping tolerances ω_*, η_*, the tolerance sequences being updated following [38, Algorithm 14.4.2]. The threshold of the ε-active sets is $\varepsilon_K = \min(\varepsilon_{\max}, \omega_K)$ at outer iteration K, for a constant $\varepsilon_{\max} > 0$; the convergence analysis requires $\varepsilon_{\max} \leq \bar{\varepsilon}$, a threshold given by Lemma 1 that only depends on Ω, and which ε_K eventually satisfies as ω_K tends to zero. Outside Assumption 4, when the ε-active bounds and the rows of A are linearly dependent, the multipliers of the decomposition (12) are not unique, and those of the least-squares solution may have the wrong sign at a first-order critical point. Our implementation then uses the multipliers that minimize the sum of the squared violations of their signs, which vanishes if and only if multipliers of the right sign exist. The constants driving the inner minimization of Algorithm 2 are tied to the subproblem criticality tolerance, so that at outer iteration K we set $\kappa_{\text{cg}} = \kappa_{\text{mlt}} = \omega_K$, while the safeguard constant of (34) is $\kappa_{\text{sds}} = 10^{-8}$, the value suggested in [2, Section 6.2].

At the start of each outer iteration, the initial approximation of the second-order terms of the Hessian is $B_0 = 0$, and the hybrid switching scheme (35) is monitored through a constant $\kappa_{\text{hyb}} \in (0, 1)$, as in [27].

We now discuss the trust-region handling. The initial radius value for Algorithm 2 is set to

$$\Delta_0 = \delta_0 \|g_0\|_\infty, \quad \delta_0 > 0.$$

The mechanism to update the trust region given in (21) still leaves important flexibility. We choose to follow a standard strategy similar to the one presented in Chapter 17 of [38]. We also added a refinement to handle the case where the ratio ρ_k is negative, which can happen when there is very poor agreement between the function and the model. In such cases, we reduce the radius more severely by taking

$$\Delta_{k+1} = \gamma_1 \Delta_k.$$

The complete update rule for the trust-region radius is then

$$\Delta_{k+1} = \begin{cases} \max\left(\alpha_2 \|s_k\|_\infty, \Delta_k\right) & \text{if } \rho_k \geq \eta_2 \\ \Delta_k & \text{if } \rho_k \in [\eta_1, \eta_2) \\ \alpha_1 \|s_k\|_\infty & \text{if } \rho_k \in [0, \eta_1) \\ \min\left(\alpha_1 \|s_k\|_\infty, \gamma_1 \Delta_k\right) & \text{if } \rho_k < 0, \end{cases}$$

whose constants satisfy $0 < \alpha_1 < 1 < \alpha_2$, $0 < \eta_1 \leq \eta_2 < 1$ and $0 < \gamma_1 < 1$, refining the generic scheme (21). The specific numerical values of all parameters used in our tests are reported in Section 4.

3 Convergence analysis

In this section, we study the convergence of Algorithm 1 towards a first-order critical point of problem (1). We start by showing global convergence of Algorithm 2, implying that the inner minimization phase of Algorithm 1 is always well defined. We then establish global convergence of the main algorithm. Our analysis of the outer iterations is based on the criticality test (8) of [37], which differs from the measure (13) used in our implementation. At the end of this section, we show that the latter certifies the former (Lemma 10), so that our convergence results apply when (15) is used as an inner iteration termination condition instead of (8).

3.1 Global convergence of the inner minimization

In this section, λ and μ are fixed, $\varphi(x) = \Phi(x, \lambda, \mu)$, and we write $\pi_\varepsilon(x) = \pi_\varepsilon(x, \nabla\varphi(x))$. We analyze Algorithm 2 with the Cauchy step computed by the procedure of Appendix A for a fixed threshold $\varepsilon > 0$ of the ε-active sets, and with the stopping test $\pi_\varepsilon(x_k) \leq \omega$.

When Ω is a box, the procedure follows the projected gradient path (22) and the analysis of [34] applies. Over a general polyhedron, the active sets along $P_\Omega[x - tg] - x$ need not be nested. For instance, let $\Omega = \{x \in \mathbb{R}^3 \mid x_1 + x_2 + x_3 = 0,\ x_1 \geq -1,\ x_2 \leq 1\}$, $x = (-0.5, -2, 2.5)$ and $g = (0.3, -0.8, 0.5)$. The bound $x_1 \geq -1$ becomes active at $t = 5/3$, the projection stays at the vertex $(-1, 1, 0)$ for t between about 4.24 and 10, and the bound $x_1 \geq -1$ is then released, the projection moving along the face $x_2 = 1$. Along this path, the reduced gradient vanishes at the vertex and has norm $\sqrt{2}/10$ afterwards. Our procedure follows instead a path along which a bound is never released once it is reached, and the analysis below is carried out for that path. The nesting of the active sets, which does not hold for the projection, then holds by construction. The price to pay is that the decrease guaranteed by the Cauchy step can no longer be related to the projected gradient: it is related to the measure π_ε, and the threshold ε is what keeps the first segment of the path from being arbitrarily short. Treating as active the bounds that are nearly active is a classical device, from the anti-zigzagging rules of [51] and the convergence analysis of feasible direction methods [54] to the projected Newton methods of [55], and it corresponds to the dominant constraints of [37] on which our criticality test is based.

We first make the standard assumption

Assumption 5 *The set $\mathcal{X} = \{x \mid \varphi(x) \leq \varphi(x_0)\} \cap \Omega$ is non-empty and compact.*

By Assumption 1, φ is twice continuously differentiable, so that its Hessian is bounded on every compact set. We recall that the quadratic model is of the form

$$q_k(s) = \frac{1}{2}\langle s, H_k s\rangle + \langle g_k, s\rangle\,,$$

where $g_k = \nabla\varphi(x_k)$ and H_k is an approximation of the Hessian based on the structured SR1 formula from Section 2.4. As in our implementation, H_k is only modified after a successful iteration, since the pair (s_k, y_k) of Section 2.4 involves the gradient at x_k+s_k. The norm used is the induced norm on matrices, and we define the nondecreasing sequence

$$b_k = 1 + \max_{0\le i\le k} \|H_i\|\,.$$

We do not require the approximations to be bounded, but only that their norms do not grow too fast compared with the decrease of the function values.

Assumption 6

$$\lim_{k\to\infty} b_k(\varphi_{k+1} - \varphi_k) = 0.$$

This assumption holds, in particular, when the approximations H_k are bounded, since the sequence $(\varphi_k)_k$ is then nonincreasing and bounded below on $\mathcal{X}$.

Threshold of the ε-active sets

For a set S of bounds, each identified by an index and a side, we write Ω_S for the face of Ω on which all the bounds of S are active. The next lemma gathers the properties of the ε-active sets that we need. They only depend on Ω.

Lemma 1 *Suppose that Assumption 4 holds. There is a constant $\bar\varepsilon \in (0, +\infty]$, depending only on A, b, l and u, such that, for all $\varepsilon \in [0, \bar\varepsilon]$ and $x \in \Omega$, with $S = \mathcal{A}_\varepsilon(x)$:*

(i) the face Ω_S is non-empty;

(ii) the matrix $\tilde A_\varepsilon$ formed by A and the rows $e_i^\top$, $i \in S$, has full row rank;

(iii) if $l_i < u_i$, the variable x_i is ε-active at one of its bounds at most, and lies then at distance at least 3ε from the other one.

Proof For a bound β, let $r_\beta(x)$ denote the gap $x_i - l_i$ or $u_i - x_i$, and for a set S of bounds, let $\rho_S = \min_{x\in\Omega} \max_{\beta\in S} r_\beta(x)$. This is the optimal value of a feasible linear program bounded below by zero, so that it is attained, and $\rho_S = 0$ if and only if $\Omega_S \neq \emptyset$. Let

$$\bar\varepsilon = \frac{1}{4}\min\left(\{\rho_S \mid \Omega_S = \emptyset\} \cup \left\{\tfrac{1}{2}(u_i - l_i) \mid -\infty < l_i < u_i < +\infty\right\}\right),$$

a minimum over a finite set of positive numbers (with $\min\emptyset = +\infty$). (i) If $S = \mathcal{A}_\varepsilon(x)$, then $\max_{\beta\in S} r_\beta(x) \le \varepsilon < \rho_S$ whenever $\Omega_S = \emptyset$, which contradicts the definition of ρ_S. (ii) For $y \in \Omega_S$, the bounds of S are active at y, and Assumption 4 at y gives the linear independence of the rows of A and of the $e_i^\top$, $i \in \mathcal{A}(y) \supseteq S$. (iii) If $l_i < u_i$ are finite, then $u_i - l_i \ge 8\bar\varepsilon$, so that $x_i - l_i \le \varepsilon \le \bar\varepsilon$ implies $u_i - x_i \ge 7\bar\varepsilon \ge 3\varepsilon$, and conversely. □

From now on, we assume that $\varepsilon \in (0, \bar{\varepsilon}]$. For $i \in \mathcal{A}_\varepsilon^-(x) \setminus \mathcal{A}_\varepsilon^+(x)$ (resp. $\mathcal{A}_\varepsilon^+(x) \setminus \mathcal{A}_\varepsilon^-(x)$), we write $n_i = e_i$ (resp. $n_i = -e_i$) for the inward normal of the ε-active bound. With the decomposition (12) of g computed with the ε-active bounds, the multiplier estimate of such a bound has the wrong sign if and only if $\theta_i e_i = -\theta_i^{\mathrm{w}} n_i$ with $\theta_i^{\mathrm{w}} > 0$.

The Cauchy path

Let $x \in \Omega$, $g = \nabla\varphi(x)$ and $\Delta > 0$. Because of the choice of the ℓ_∞ norm, the steps within the trust region are the vectors of

$$\left\{ d \in \mathbb{R}^n \mid Ad = 0,\ l^\Delta \le d \le u^\Delta \right\}, \tag{40}$$

with $l_i^\Delta = \max(-\Delta, l_i - x_i)$ and $u_i^\Delta = \min(\Delta, u_i - x_i)$ for all $i = 1, \ldots, n$. Let $\mathcal{A}_\varepsilon = \mathcal{A}_\varepsilon(x)$ and $\mathcal{V} = \{i \in \mathcal{A}_\varepsilon \mid \theta_i^{\mathrm{w}} > 0\}$ be the set of ε-active bounds whose multiplier estimate has the wrong sign. For $F \subseteq \mathcal{V}$, let $T_F = \{v \mid Av = 0,\ v_i = 0 \text{ for } i \in \mathcal{A}_\varepsilon \setminus F\}$. The procedure of Appendix A sets $F_0 = \mathcal{V}$ and, for $j = 0, 1, \ldots$,

$$F_{j+1} = F_j \setminus \left\{ i \in F_j \mid \left\langle n_i, P_{T_{F_j}}[-g] \right\rangle < 0 \right\}, \tag{41}$$

until $F_{j+1} = F_j =: F$. The variables of $\mathcal{A}(0) := \mathcal{A}_\varepsilon \setminus F$ are fixed at their current values, and $z_0 = P_{T_F}[g]$. The path is then defined by $s(0) = 0$ and

$$s(t) = s(t_i) - (t - t_i) z_i \quad \text{for } t \in [t_i, t_{i+1}),$$

where

$$z_i = P_{T_i}[g], \qquad T_i = \{v \mid Av = 0,\ v_j = 0 \text{ for } j \in \mathcal{A}(t_i)\}, \tag{42}$$

t_{i+1} is the first value of t at which a free component reaches its bound in (40), and $\mathcal{A}(t_{i+1})$ is obtained by adding the corresponding indices to $\mathcal{A}(t_i)$. The Cauchy step $s^C = s(t^C)$ is obtained at the first value t^C at which $\psi(t) = q(s(t))$ satisfies $\psi'(t_+^C) \ge 0$, or at which $z_i = 0$. The minor iterations of Section 2.3 then provide a step s_k that remains in (40) and satisfies

$$q_k(s_k) \le q_k(s_k^C). \tag{43}$$

Proposition 2 *Suppose that Assumption 4 holds. The path $s(t)$ has the following properties.*

(i) It is piecewise linear, lies in (40), and has finitely many breakpoints.

(ii) The working sets are nested, $\mathcal{A}(t) \subseteq \mathcal{A}(t')$ for $0 < t \le t'$, so that $T_{i+1} \subseteq T_i$ and $\|z_{i+1}\| \le \|z_i\|$.

(iii) $s(t) = -\int_0^t z(\tau)\, d\tau$, $\langle g, z_i \rangle = \|z_i\|^2$ and $\|s(t)\| \le t\|z_0\| \le t\|g\|$.

(iv) $-z_0$ is a feasible descent direction at x: $Az_0 = 0$, $(z_0)_i = 0$ for $i \in \mathcal{A}(0)$, and $\langle n_i, -z_0 \rangle \ge 0$ for $i \in F$.

Proof The nesting (ii) holds by construction, since a bound added to the working set is never released along the path; projecting g onto a smaller subspace reduces the norm of the projection. Each breakpoint of positive length adds at least one index to the working set, so

that there are at most n of them. Statements (i) and (iii) follow, $\langle g, z_i\rangle = \|z_i\|^2$ because z_i is an orthogonal projection of g. Each segment remains in (40) up to the next breakpoint by definition of the latter. Statement (iv) is the termination condition of (41). □

Initial reduced gradient

The next lemma is the key result of this section. It replaces, for the path we compute, the comparison between the reduced gradient along the projected gradient path and the norm of the projected gradient which is used in the bound-constrained case.

Lemma 3 *Suppose that Assumption 4 holds. There is a constant $\kappa_0 \in (0,1]$, depending only on A and on the bounds, such that, for all $x \in \Omega$ and $\varepsilon \in (0, \bar{\varepsilon}]$,*

$$\|z_0\| \geq \kappa_0 \, \pi_\varepsilon(x).$$

Proof Let $u = P_{T_\varepsilon(x)}[g]$ and $\omega_i = \theta_i^{\mathrm{w}} > 0$ for $i \in \mathcal{V}$. For $F' \subseteq \mathcal{V}$, the rows of A and the vectors e_i, $i \in \mathcal{A}_\varepsilon \setminus F'$, are orthogonal to $T_{F'}$, while $P_{T_{F'}}[u] = u$ since $T_\varepsilon(x) \subseteq T_{F'}$. With $\theta_i e_i = -\omega_i n_i$ for $i \in F'$, the decomposition (12) of g gives

$$P_{T_{F'}}[g] = u - \sum_{i \in F'} \omega_i \, P_{T_{F'}}[n_i]\,.$$

Moreover $\langle u, P_{T_{F'}}[n_i]\rangle = \langle u, n_i\rangle = 0$ since $u_i = 0$ for $i \in \mathcal{A}_\varepsilon$. Denoting by $G(F')$ the Gram matrix of the vectors $P_{T_{F'}}[n_i]$, $i \in F'$, we obtain

$$\big\|P_{T_{F'}}[g]\big\|^2 = \|u\|^2 + \omega_{F'}^\top G(F')\,\omega_{F'}, \qquad \big\langle n_i, P_{T_{F'}}[-g]\big\rangle = \big(G(F')\,\omega_{F'}\big)_i \quad (i \in F'). \tag{44}$$

The matrix $G(F')$ is positive definite. Indeed, if $\sum_{i\in F'} c_i P_{T_{F'}}[n_i] = 0$, then $\sum_i c_i n_i$ belongs to $T_{F'}^\perp$, which is spanned by the rows of A and the e_j, $j \in \mathcal{A}_\varepsilon \setminus F'$, and $c = 0$ by Lemma 1(ii).

By (44), the iteration (41) reads $F_{j+1} = F_j \setminus \{i \in F_j \mid (G(F_j)\,\omega_{F_j})_i < 0\}$. It only depends on ω, and is invariant when ω is multiplied by a positive scalar. We show that there is $c > 0$ such that $\|\omega_F\| \geq c\,\|\omega\|$ for all $\omega \in (0,\infty)^{\mathcal{V}}$. The sequence of sets $\sigma = (F_0 \supsetneq F_1 \supsetneq \cdots \supsetneq F_J = F)$ produced by (41) takes finitely many values, so that it suffices to consider a fixed σ. Assume that there are ω^ℓ producing σ, with $\|\omega^\ell\| = 1$ and $\omega^\ell_F \to 0$. Up to a subsequence, $\omega^\ell \to \bar{\omega} \geq 0$ with $\|\bar{\omega}\| = 1$ and $\bar{\omega}_F = 0$. We show by backward induction that $\bar{\omega}$ vanishes on F_j for all j. Assume that it vanishes on F_{j+1}, and let $R = F_j \setminus F_{j+1}$. For $i \in R$, $(G(F_j)\,\omega^\ell_{F_j})_i < 0$ for all ℓ, hence $(G_{RR}\,\bar{\omega}_R)_i \leq 0$ in the limit, where G_{RR} is the principal block of $G(F_j)$ indexed by R. Multiplying by $\bar{\omega}_i \geq 0$ and summing over $i \in R$ yields $\bar{\omega}_R^\top G_{RR}\,\bar{\omega}_R \leq 0$, so that $\bar{\omega}_R = 0$ since G_{RR} is positive definite. For $j = 0$, this gives $\bar{\omega} = 0$, a contradiction.

Therefore, by (44),

$$\|z_0\|^2 = \|u\|^2 + \omega_F^\top G(F)\,\omega_F \geq \|u\|^2 + \lambda_{\min}(G(F))\,c^2\,\|\theta^{\mathrm{w}}\|^2 \geq \kappa_0^2 \max\big(\|u\|_\infty, \|\theta^{\mathrm{w}}\|_\infty\big)^2,$$

where $\kappa_0 = \min\Big(1, c\sqrt{\lambda_{\min}(G(F))}\Big)$, the minimum being taken over the finitely many possible sets $\mathcal{A}_\varepsilon$, $\mathcal{V}$ and F. □

Releasing all the bounds of the wrong sign at once, and then fixing again those along which the direction points outwards, is therefore safe: the final set F is non-empty as soon as $\mathcal{V}$ is, and it retains a uniform fraction of the violation of the sign conditions.

Guaranteed decrease

Theorem 4 *Suppose that Assumption 4 holds and that $\varepsilon \in (0, \bar{\varepsilon}]$. Then, at every iteration k,*

$$-q_k(s_k) \geq \frac{\kappa_0 \, \pi_\varepsilon(x_k)}{2} \min\left(\varepsilon, \ \Delta_k, \ \frac{\kappa_0 \, \pi_\varepsilon(x_k)}{2(1 + \|H_k\|)}\right). \tag{45}$$

Proof If $z_0 = 0$, then $\pi_\varepsilon(x_k) = 0$ by Lemma 3, and there is nothing to prove. Otherwise, let $t_1 > 0$ be the first breakpoint of positive length. On $[0, t_1)$, $s(t) = -tz_0$, so that $|s_j(t)| \leq t\|z_0\|$ for every component j. The components fixed in $\mathcal{A}(0)$ do not move. A free component that is not ε-active lies at a distance larger than ε from its bounds. A released component moves inwards by Proposition 2(iv), and lies at a distance at least 3ε from its other bound by Lemma 1(iii). The trust-region bounds lie at distance Δ_k. Hence $t_1 \geq \min(\varepsilon, \Delta_k)/\|z_0\|$.

On $[0, t_1)$, $\psi(t) = -t\|z_0\|^2 + \frac{t^2}{2}\langle z_0, H_k z_0\rangle$, so that $\psi'(t) \leq -\|z_0\|^2(1 - t\|H_k\|) \leq -\frac{1}{2}\|z_0\|^2$ for $t \leq \hat{t} := \min(t_1, 1/(2(1 + \|H_k\|)))$. No termination condition of the path can thus be met before $\hat{t}$, so that $t^C \geq \hat{t}$. Since ψ is decreasing on $[0, t^C]$, we obtain $\psi(t^C) \leq \psi(\hat{t}) \leq -\frac{1}{2}\|z_0\|^2\hat{t}$, that is

$$-q_k(s_k^C) \geq \min\left(\frac{\|z_0\|}{2}\min(\varepsilon, \Delta_k), \ \frac{\|z_0\|^2}{4(1 + \|H_k\|)}\right),$$

and we conclude with Lemma 3 and (43). □

The threshold ε cannot be omitted from (45). With $\varepsilon = 0$, the same argument only gives $t_1 \geq \min(\delta(x_k), \Delta_k)/\|z_0\|$, where $\delta(x_k)$ is the distance from x_k to the nearest inactive bound, and this distance may be arbitrarily small. Consider again the polyhedron of the example above, with the linear model $q(s) = \langle g, s\rangle$, $g = (0.3, -0.8, 0.5)$, $\Delta = 1$, and $x = (-1, 1, 0) + \delta(1, -3, 2)$. No bound is active at x, so that $z_0 = g$, $\pi_0(x) = 0.8$ and $\delta(x) = \delta$. The path reaches the bound $x_1 \geq -1$ at $t = 10\delta/3$, then the vertex $(-1, 1, 0)$, where the working set leaves no degree of freedom, and the decrease of the model is $3.7\,\delta$, whereas $\pi_0(x) = 0.8$ and $\|P_\Omega[x - g] - x\| \to \sqrt{2}/10$ as $\delta \to 0$. No bound of the form (45) without ε, nor of the form $\kappa\, h \min(h/b_k, \Delta_k)$ with h the norm of the projected gradient, can thus hold for the path we compute. The mechanism is that of the zigzagging phenomenon identified by Zoutendijk for feasible direction methods [51]: in the example of [52], steepest ascent restricted by the active nonnegativity constraints converges, with exact line searches, to a non-stationary point of a concave function, every step terminating on a constraint that was nearly active. Wolfe's example requires unbounded second derivatives, whereas ours concerns the decrease provided by a single Cauchy step of a quadratic model, which is what the trust-region analysis requires. Over the nonnegative orthant or a box, following the projected path [60] avoids the phenomenon, and our path coincides with it; over a polyhedron, it does not. We use instead the remedy proposed in [51, Chapter 7]: with $\varepsilon > 0$, the bounds within distance ε of such a point are treated as active, and the decrease is bounded below independently of δ.

Convergence of the inner iterations

Theorem 5 *Suppose that Assumptions 1, 4, 5 and 6 hold, and that $\varepsilon \in (0, \bar{\varepsilon}]$. If Algorithm 2 generates an infinite sequence of iterates, then*

$$\liminf_{k \to \infty} \pi_\varepsilon(x_k) = 0.$$

Consequently, for every $\omega > 0$, the stopping test $\pi_\varepsilon(x_k) \leq \omega$ is satisfied after finitely many iterations.

Proof Assume, for contradiction, that $\pi_\varepsilon(x_k) \geq \underline{\pi} > 0$ for all $k \geq k_0$. Since $1 + \|H_k\| \leq b_k$ and $b_k \geq 1$, Theorem 4 gives

$$-q_k(s_k) \geq c \min\left(\Delta_k, \frac{\zeta}{b_k}\right), \qquad c = \frac{\kappa_0 \underline{\pi}}{2}, \quad \zeta = \min\left(\varepsilon, \frac{\kappa_0 \underline{\pi}}{2}\right), \tag{46}$$

for all $k \geq k_0$. Let L bound $\|\nabla^2 \varphi\|$ on the compact set $\{x + s \mid x \in \mathcal{X},\ \|s\|_\infty \leq 1\}$, and $L' = n(L+1)/2$. If $\|s_k\|_\infty \leq 1$, Taylor's theorem gives

$$|\varphi(x_k + s_k) - \varphi(x_k) - q_k(s_k)| \leq \tfrac{1}{2}\,(L + \|H_k\|)\,\|s_k\|^2 \leq L' b_k \|s_k\|_\infty^2.$$

Let $c_1 = \min\left(1, \zeta, (1-\eta_1)c/(2L')\right)$ and $c_2 = \min\left(c_1, \sqrt{(1-\eta_1)c\,c_1/L'}\right)$. At an unsuccessful iteration, $\rho_k < \eta_1$ implies $(1-\eta_1)(-q_k(s_k)) < \varphi(x_k + s_k) - \varphi(x_k) - q_k(s_k)$.

First, if iteration $k \geq k_0$ is unsuccessful, then $\Delta_k > c_1/b_k$ and $\|s_k\|_\infty \geq c_2/b_k$. Indeed, if $\Delta_k \leq c_1/b_k$, then $\|s_k\|_\infty \leq \Delta_k \leq 1$ and $\min(\Delta_k, \zeta/b_k) = \Delta_k$, so that $(1-\eta_1)\,c\,\Delta_k < L' b_k \Delta_k^2$, that is $\Delta_k > (1-\eta_1)c/(L' b_k) \geq 2c_1/b_k$, a contradiction. Next, if $\|s_k\|_\infty \geq 1$, then $\|s_k\|_\infty \geq c_2/b_k$. Otherwise, since $\min(\Delta_k, \zeta/b_k) \geq c_1/b_k$, we obtain $L' b_k \|s_k\|_\infty^2 > (1-\eta_1)c\,c_1/b_k$, hence $\|s_k\|_\infty \geq c_2/b_k$.

Second, $\Delta_k \geq c_3/b_k$ for all $k \geq k_0$, with $c_3 = \min(b_{k_0}\Delta_{k_0}, \alpha_1 c_2, \gamma_1 c_1)$. This holds at k_0. If it holds at a successful iteration k, then $\Delta_{k+1} \geq \Delta_k \geq c_3/b_k \geq c_3/b_{k+1}$. At an unsuccessful iteration, the radius update of Section 2.5 gives $\Delta_{k+1} \geq \min(\alpha_1 \|s_k\|_\infty, \gamma_1 \Delta_k) \geq \min(\alpha_1 c_2, \gamma_1 c_1)/b_k$, and $b_{k+1} = b_k$ since H_k is not modified.

Finally, if there are finitely many successful iterations, then x_k, H_k and $b_k = \bar{b}$ are eventually constant, and $\Delta_{k+1} \leq \alpha_1 \Delta_k$ for all large k, which contradicts $\Delta_k \geq c_3/\bar{b}$. Otherwise, for every successful iteration $k \geq k_0$,

$$b_k\,(\varphi_k - \varphi_{k+1}) \geq b_k\,\eta_1\,(-q_k(s_k)) \geq \eta_1\,c\,\min(c_3, \zeta) > 0,$$

by (46), which contradicts Assumption 6 since there are infinitely many such iterations. □

Theorem 5 only establishes that the measure π_ε becomes small along a subsequence, which is what the termination of the inner iterations requires. Unlike the projected gradient, π_ε is not continuous with respect to x, and we do not claim that $\pi_\varepsilon(x_k) \to 0$. The link between π_ε and the criticality test (8) used in the analysis of the outer iterations is established in Lemma 10.

3.2 Global convergence of the algorithm

We now establish that every limit point of the outer iteration sequence is a first-order critical point of (1), for a feasible set combining nonlinear equalities with a general polyhedron. Our problem is a particular case of the one considered in [37], whose analysis also covers linear equalities alongside the linear inequalities, as noted there, and our criticality test (8) is theirs. We restate the main steps of their argument in our notation, for completeness and because the inner iteration of Section 3.1 is designed to satisfy this test. We first make the following assumption about the iterates considered.

Assumption 7 *The iterates produced by Algorithm 1 lie within a closed bounded domain of $\mathbb{R}^n$.*

Let $(x_K)_K$ be a sequence of such iterates. Assumption 7 then ensures the existence of a subsequence indexed by $\mathcal{K} \subseteq \mathbb{N}$ converging to a limit point x_*. Since the iterates satisfy $x_K \in \Omega$ for all K, we also have $x_* \in \Omega$. We recall from (9) that $\mathcal{A}(x)$ is the set of bounds active at x and $T(x)$ the associated tangent space, and we abbreviate $\mathcal{A}_* := \mathcal{A}(x_*)$. We denote by $\tilde{A}_*$ the matrix gathering the linear constraints active at x_*,

$$\tilde{A}_* := \begin{pmatrix} A \\ E_*^\top \end{pmatrix},$$

where E_* is the submatrix of the identity whose columns are e_i, $i \in \mathcal{A}_*$. Consistently with the well-posedness of the projector (11), $\tilde{A}_*$ has full row rank, so $T(x_*) = \text{null}(\tilde{A}_*)$ and there is an orthonormal matrix N_* whose columns form a basis of $T(x_*)$. The next assumption ensures that the null space of the active linear constraints is large enough to achieve feasibility of the nonlinear constraints, and that the gradients of those constraints are linearly independent within that space.

Assumption 8 *The rank of the matrix $C(x_*)N_*$ is no smaller than n_c at any limit point x_* of the sequence $(x_K)_K$.*

We now introduce the remaining notation and notions used in our convergence analysis.

Since Ω is of the form (2), the normal cone to Ω at $x \in \Omega$ can be defined as

$$\mathcal{N}_\Omega(x) = \left\{ A^\top \xi + \sum_{i \in \mathcal{A}^+(x)} \sigma_i^+ e_i - \sum_{i \in \mathcal{A}^-(x)} \sigma_i^- e_i \;\middle|\; \xi \in \mathbb{R}^m,\ \sigma_i^+, \sigma_i^- \geq 0 \right\}, \tag{47}$$

where the equality multipliers ξ are free in sign while the bound multipliers are signed. The first-order criticality condition $P_\Omega\left[x_* - \nabla_x \mathcal{L}(x_*, \lambda_*)\right] = x_*$ of problem (1) is then equivalent to

$$-\nabla_x \mathcal{L}(x_*, \lambda_*) \in \mathcal{N}_\Omega(x_*).$$

Finally, for vectors x at which the matrix $C(x)N_*N_*^\top C(x)^\top$ is non-singular, the least-squares multipliers associated with $\tilde{A}_*$ are defined by

$$\lambda(x) = -\big((C(x)N_*)^\dagger\big)^\top N_*^\top \nabla f(x), \tag{48}$$

where $(C(x)N_*)^\dagger = N_*^\top C(x)^\top \big(C(x)N_*N_*^\top C(x)^\top\big)^{-1}$ is the right pseudoinverse of $C(x)N_*$. When $(C(x)N_*)^\dagger$ is well defined, Assumption 1 ensures that $\lambda(x)$ is differentiable and that its Jacobian $\nabla\lambda(x)$ is bounded in a neighborhood of x_* (see [37, Lemma 2.1]).

We can now proceed with the global convergence proof. The first lemma is the counterpart of [37, Lemma 4.1]. It shows that the criticality test (8) controls the reduced gradient at the limit point.

Lemma 6 *Let $(x_K)_{K\in\mathcal{K}} \subseteq \Omega$ be a sequence converging to x_* and such that*

$$\left\| P_{\mathcal{T}_{\varepsilon_K}(x_K)}\left[-\nabla_x \Phi_K\right]\right\| \leq \omega_K,$$

with $0 \leq \varepsilon_K \leq \omega_K$ and $\lim_{K\to\infty} \omega_K = 0$. Then

$$\left\| P_{T(x_*)}\left[\nabla_x \Phi_K\right]\right\| \leq \omega_K,$$

for all $K \in \mathcal{K}$ large enough.

Proof We write $\mathcal{T}_K = \mathcal{T}_{\varepsilon_K}(x_K)$ and $\mathcal{N}_K = \mathcal{N}_{\varepsilon_K}(x_K)$. A bound that is inactive at x_* has a positive gap at x_*, hence a gap larger than ε_K at x_K for $K \in \mathcal{K}$ large enough, since $x_K \to x_*$ and $\varepsilon_K \to 0$. Conversely, the gap at x_* of a bound that is ε_K-active at x_K for infinitely many K is zero. Hence, for $K \in \mathcal{K}$ large enough, the ε_K-active bounds at x_K are active at x_*, on the same side, and the subspace $T(x_*)$ defined in (9) is contained in

$$V_K = \left\{ d \in \mathbb{R}^n \mid Ad = 0,\ d_i = 0 \text{ for } i \in \mathcal{A}_{\varepsilon_K}(x_K) \right\},$$

which is itself contained in $\mathcal{T}_K$. Since V_K is a subspace contained in $\mathcal{T}_K$, every $w \in \mathcal{N}_K$ satisfies $\langle w, v\rangle \leq 0$ and $\langle w, -v\rangle \leq 0$ for all $v \in V_K$, that is, $\mathcal{N}_K$ is orthogonal to V_K. The Moreau decomposition $-\nabla_x \Phi_K = P_{\mathcal{T}_K}\left[-\nabla_x \Phi_K\right] + P_{\mathcal{N}_K}\left[-\nabla_x \Phi_K\right]$ then gives $P_{V_K}\left[-\nabla_x \Phi_K\right] = P_{V_K}\left[P_{\mathcal{T}_K}\left[-\nabla_x \Phi_K\right]\right]$, so that

$$\left\| P_{T(x_*)}\left[\nabla_x \Phi_K\right]\right\| \leq \left\| P_{V_K}\left[\nabla_x \Phi_K\right]\right\| \leq \left\| P_{\mathcal{T}_K}\left[-\nabla_x \Phi_K\right]\right\| \leq \omega_K,$$

the first inequality following from $T(x_*) \subseteq V_K$. □

The next lemma is a standard result about the behavior of the Lagrange multipliers in AL methods.

Lemma 7 ([38], Lemma 14.4.1) *Assume that $\lim_{K\to\infty} \mu_K = \infty$ when Algorithm 1 is executed. Then*

$$\lim_{K\to\infty} \frac{\|\lambda_K\|}{\mu_K} = 0.$$

In [38], the penalty parameter tends to zero and the lemma requires a bound on the sums of the tolerances η_K between two increases of the penalty parameter, which the updates of Algorithm 1, those of [38, Algorithm 14.4.2], satisfy because $\mu_K \geq \mu_0 > 1$.

We now state the main convergence results. Their statement parallels [37, Lemma 4.4 and Theorem 4.6]. Since the framework presented there differs from ours in the criticality measure used, the arguments of the proof must be adapted.

Lemma 8 *Suppose that Assumptions 1 and 3 hold. Let $(x_K)_{K\in\mathcal{K}}$ be a sequence of iterates in Ω satisfying Assumption 7 and converging to x_* for which Assumption 8 holds. Let $\lambda_* = \lambda(x_*)$, $(\lambda_K)_{K\in\mathcal{K}}$ be any sequence of vectors and $(\mu_K)_{K\in\mathcal{K}}$ be a non-decreasing sequence of positive scalars. Assume that (8) holds with $\lim_{K\in\mathcal{K}} \omega_K = 0$.*

Then, there are positive constants κ_2, κ_3 such that

$$\|\bar{\lambda}_K - \lambda_*\| \leq \kappa_2 \omega_K + \kappa_3 \|x_K - x_*\|, \tag{49}$$

$$\|\lambda(x_K) - \lambda_*\| \leq \kappa_3 \|x_K - x_*\|, \tag{50}$$

and

$$\|c(x_K)\| \leq \kappa_2 \frac{\omega_K}{\mu_K} + \kappa_3 \frac{\|x_K - x_*\|}{\mu_K} + \frac{\|\lambda_K - \lambda_*\|}{\mu_K}. \tag{51}$$

If, in addition, $c(x_*) = 0$, then x_* is a first-order critical point for problem (1) with corresponding Lagrange multipliers λ_* and with $\lim_{K \in \mathcal{K}} \bar{\lambda}_K = \lim_{K \in \mathcal{K}} \lambda(x_K) = \lambda_*$.

Proof By Assumptions 1 and 8, for K sufficiently large, the matrix $(C_K N_*)^\dagger$ is well defined, bounded and converges to $(C_* N_*)^\dagger$. Therefore, there is a constant κ_2 such that

$$\|((C_K N_*)^\dagger)^\top\| \leq \kappa_2.$$

Furthermore, by Lemma 6, we have

$$\|N_*^\top \nabla_x \Phi_K\| = \left\| P_{T(x_*)} \left[\nabla_x \Phi_K \right] \right\| \leq \omega_K,$$

for $K \in \mathcal{K}$ large enough. Thus we may deduce

$$\begin{aligned} \|\bar{\lambda}_K - \lambda(x_K)\| &= \|((C_K N_*)^\dagger)^\top N_*^\top \nabla f_K + \bar{\lambda}_K\| \\ &= \|((C_K N_*)^\dagger)^\top (N_*^\top \nabla f_K + (C_K N_*)^\top \bar{\lambda}_K)\| \\ &= \|((C_K N_*)^\dagger)^\top (N_*^\top \nabla_x \Phi_K)\| \\ &\leq \|((C_K N_*)^\dagger)^\top\| \, \|N_*^\top \nabla_x \Phi_K\| \\ &\leq \kappa_2 \omega_K. \end{aligned} \tag{52}$$

Furthermore, by the mean value theorem for vector-valued functions, we have

$$\lambda(x_K) - \lambda_* = \int_0^1 \nabla \lambda(x(t))(x_K - x_*) dt,$$

where $x(t) = x_K + t(x_* - x_K)$. By our discussion of the least-squares multipliers (48), one can show that the above integrand is bounded for x_K sufficiently close to x_*, hence

$$\|\lambda(x_K) - \lambda_*\| \leq \kappa_3 \|x_K - x_*\|,$$

for $K \in \mathcal{K}$ sufficiently large and for some positive constant κ_3, giving (50). Then, combining the latter inequality and (52), we may deduce that

$$\begin{aligned} \|\bar{\lambda}_K - \lambda_*\| &\leq \|\bar{\lambda}_K - \lambda(x_K)\| + \|\lambda(x_K) - \lambda_*\| \\ &\leq \kappa_2 \omega_K + \kappa_3 \|x_K - x_*\|, \end{aligned}$$

which is inequality (49). Next, by expression (4), we have $c(x_K) = (\bar{\lambda}_K - \lambda_K)/\mu_K$, which leads to

$$\begin{aligned} \|c(x_K)\| &\leq \frac{\|\bar{\lambda}_K - \lambda_*\|}{\mu_K} + \frac{\|\lambda_K - \lambda_*\|}{\mu_K} \\ &\leq \kappa_2 \frac{\omega_K}{\mu_K} + \kappa_3 \frac{\|x_K - x_*\|}{\mu_K} + \frac{\|\lambda_K - \lambda_*\|}{\mu_K}, \end{aligned}$$

where we have used (49). This concludes the first part of the lemma.

Since $(\omega_K)_K$ tends to 0 and $(x_K)_{K \in \mathcal{K}}$ converges to x_* by assumption, inequalities (49) and (50) imply that both sequences $(\bar{\lambda}_K)_K$ and $(\lambda(x_K))_K$ converge to λ_* for $K \in \mathcal{K}$. Therefore, from identity (5), we obtain that the sequence of gradients $\nabla_x \Phi_K$ converges to $\nabla_x \mathcal{L}(x_*, \lambda_*)$.

Finally, assume that $c(x_*) = 0$. By the Moreau decomposition, $-\nabla_x \Phi_K = P_{\mathcal{T}_K}[-\nabla_x \Phi_K] + w_K$, where $\mathcal{T}_K = \mathcal{T}_{\varepsilon_K}(x_K)$ and $w_K \in \mathcal{N}_K = \mathcal{N}_{\varepsilon_K}(x_K)$, and $\|P_{\mathcal{T}_K}[-\nabla_x \Phi_K]\| \leq \omega_K$ by (8). As in the proof of Lemma 6, the ε_K-active bounds at x_K are active at x_*, on the same side, for $K \in \mathcal{K}$ large enough, so that $\mathcal{N}_K \subseteq \mathcal{N}_\Omega(x_*)$ by (47). Since $\nabla_x \Phi_K \to \nabla_x \mathcal{L}(x_*, \lambda_*)$ and $\omega_K \to 0$, we obtain $w_K \to -\nabla_x \mathcal{L}(x_*, \lambda_*)$, and the normal cone $\mathcal{N}_\Omega(x_*)$ being closed,

$$-\nabla_x \mathcal{L}(x_*, \lambda_*) \in \mathcal{N}_\Omega(x_*).$$

Therefore, we have established that (x_*, λ_*) is a KKT point and the lemma is proved. □

We can now establish the global convergence of Algorithm 1.

Theorem 9 *Suppose that Assumptions 1 and 3 hold. Let $(x_K)_K$ be a sequence of iterates generated by Algorithm 1 satisfying Assumption 7. Further assume that there is a subsequence $(x_K)_{K \in \mathcal{K}}$ converging to x_*, for which Assumption 8 holds and let $\lambda_* = \lambda(x_*)$. Then, the conclusions of Lemma 8 hold.*

Proof As in the proof of Lemma 8, the inequalities (49), (50), (51) can be derived from the assumptions. All that remains is to show that $c(x_*) = 0$. There are two cases to distinguish.

First, if the sequence of penalty parameters is bounded above, then there is K_0 sufficiently large such that $\|c(x_K)\| \leq \eta_K$ for all $K \geq K_0$. Since $\eta_K \to 0$, we get that $c(x_*) = 0$.

Next, if the sequence of penalty parameters is unbounded, then by the update rule, we have that $(\mu_K)_K$ grows to ∞. Hence, by Lemma 7, the ratio λ_K / μ_K converges to zero. Therefore, by inequality (51), we also obtain that $c(x_*) = 0$. In both cases, x_* is feasible with respect to the nonlinear constraints and the remaining conclusions of Lemma 8 hold. □

The criticality test (8) involves a projection onto the cone $\mathcal{T}_{\varepsilon_K}(x_K)$, which amounts to a quadratic program, whereas our implementation monitors criticality through the measure (13). The next lemma links the two. Its right inequality reformulates [37, Lemma 7.1] for the least-squares multiplier estimates of (12), and the reduced gradient criterion [37, (7.3)] used in MINOS and LSNNO. We state it for a generic vector v, which stands for $\nabla_x \Phi_K$, and we write $\pi_\varepsilon(x, v)$ for the right-hand side of (13) computed from the decomposition (12) of v with the ε-active bounds, and $\operatorname{dist}(w, S) = \inf_{y \in S} \|w - y\|$.

Lemma 10 *Suppose that Assumption 4 holds, let $\bar{\varepsilon}$ be the threshold of Lemma 1, and let $\sigma_{\min} > 0$ be the smallest of the smallest singular values of the matrices $\tilde{A}_\varepsilon$, over $x \in \Omega$ and $\varepsilon \in [0, \bar{\varepsilon}]$. Then, for all $\varepsilon \in [0, \bar{\varepsilon}]$, $x \in \Omega$ and $v \in \mathbb{R}^n$,*

$$\min(1, \sigma_{\min})\, \pi_\varepsilon(x, v) \leq \left\| P_{\mathcal{T}_\varepsilon(x)}[-v] \right\| = \operatorname{dist}(-v, \mathcal{N}_\varepsilon(x)) \leq \sqrt{n}\, \pi_\varepsilon(x, v). \tag{53}$$

In particular, $\pi_\varepsilon(x, v) = 0$ if and only if $-v \in \mathcal{N}_\varepsilon(x)$. Moreover, for $\varepsilon = 0$,

$$\|P_\Omega[x - v] - x\| \leq \operatorname{dist}(-v, \mathcal{N}_\Omega(x)), \tag{54}$$

so that $\pi(x, v) = 0$ if and only if x is a first-order critical point of the minimization over Ω of a function whose gradient at x is v.

Proof Write $S = \mathcal{A}_\varepsilon(x)$ and $u = P_{T_\varepsilon(x)}[v]$, and decompose $v = u + A^\top \xi + E_S\theta$ as in (12) with the ε-active bounds. Since $u \in T_\varepsilon(x)$, $u_i = 0$ for all $i \in S$, so that u is orthogonal to the row space of $\tilde{A}_\varepsilon$ and its support is disjoint from S. The equality in (53) is the Moreau decomposition of $-v$ with respect to $\mathcal{T}_\varepsilon(x)$ and its polar $\mathcal{N}_\varepsilon(x)$.

Let θ^+ be obtained from θ by setting to zero its components of the wrong sign, so that $|\theta_i - \theta_i^+| = \theta_i^{\mathrm{w}}$ for all $i \in S$, and let $w = -A^\top \xi - E_S\theta^+$. By the sign conditions satisfied by θ^+, $w \in \mathcal{N}_\varepsilon(x)$. Moreover $v + w = u + E_S(\theta - \theta^+)$, the two terms having disjoint supports, so that

$$\operatorname{dist}(-v, \mathcal{N}_\varepsilon(x)) \le \|v + w\| = \left(\|u\|^2 + \|\theta^{\mathrm{w}}\|^2\right)^{1/2} \le \sqrt{n}\max\left(\|u\|_\infty, \|\theta^{\mathrm{w}}\|_\infty\right),$$

which is the right inequality in (53). Conversely, any $w \in \mathcal{N}_\varepsilon(x)$ can be written $w = A^\top \xi' + E_S\tau$ with $\tau_i \ge 0$ for $i \in \mathcal{A}_\varepsilon^+(x) \setminus \mathcal{A}_\varepsilon^-(x)$ and $\tau_i \le 0$ for $i \in \mathcal{A}_\varepsilon^-(x) \setminus \mathcal{A}_\varepsilon^+(x)$. Then $v + w = u + \tilde{A}_\varepsilon^\top(\xi + \xi', \theta + \tau)$ and, by orthogonality,

$$\|v + w\|^2 = \|u\|^2 + \left\|\tilde{A}_\varepsilon^\top(\xi + \xi', \theta + \tau)\right\|^2 \ge \|u\|^2 + \sigma_{\min}^2\|\theta + \tau\|^2.$$

Whenever $\theta_i^{\mathrm{w}} > 0$, θ_i and τ_i have the same sign, so that $|\theta_i + \tau_i| \ge \theta_i^{\mathrm{w}}$. Hence $\|v + w\|^2 \ge \|u\|^2 + \sigma_{\min}^2\|\theta^{\mathrm{w}}\|^2 \ge \min(1, \sigma_{\min})^2\,\pi_\varepsilon(x, v)^2$ for all $w \in \mathcal{N}_\varepsilon(x)$, which is the left inequality in (53). The characterization of the case $\pi_\varepsilon(x, v) = 0$ follows, $\mathcal{N}_\varepsilon(x)$ being closed.

Finally, let $\varepsilon = 0$, so that $\mathcal{N}_0(x) = \mathcal{N}_\Omega(x)$, and let $p = P_\Omega[x - v]$ and $d = p - x$. The characterization of the projection onto the closed convex set Ω gives $\langle x - v - p, y - p\rangle \le 0$ for all $y \in \Omega$, hence $\|d\|^2 \le \langle -v, d\rangle$ by taking $y = x$. By definition of the normal cone, $\langle w, p - x\rangle \le 0$ for all $w \in \mathcal{N}_\Omega(x)$, so that $\|d\|^2 \le \langle -v - w, d\rangle \le \|v+w\|\,\|d\|$ for all $w \in \mathcal{N}_\Omega(x)$, which yields (54). □

Consequently, if $\varepsilon_{\max} \le \bar{\varepsilon}$, each inner minimization terminates by Theorem 5 at a point satisfying (15), hence (8) with ω_K replaced by $\sqrt{n}\,\omega_K$ by Lemma 10. This tolerance still tends to zero, so that Lemmas 6 and 8 and Theorem 9 apply to the iterates produced by the implementation, without any assumption on the signs of the bound multipliers. Likewise, the final test of Algorithm 1, implemented as $\pi_{\varepsilon_*}(x_K, \lambda_K, \mu_K) \le \omega_*$, certifies $\|P_{\mathcal{T}_{\varepsilon_*}(x_K)}[-\nabla_x\Phi_K]\| \le \sqrt{n}\,\omega_*$. As in [37], this certificate concerns the bounds that lie within distance $\varepsilon_* \le \omega_*$ of the final point. It could be turned into a bound on the projected gradient by Hoffman's error bound [61], but with a constant depending on Ω that is not computable in practice.

4 Numerical experiments

In this section, we report numerical experiments conducted with the Julia implementation of our algorithm, named TRAULLS[1]. All runs were performed with Julia 1.12.1 on a machine equipped with an Intel® Core™ Ultra 9 processor 285 and 64 GiB of RAM. All the benchmarks we present here are also accessible in the `benchmark` folder of the GitHub repository. The tests were carried out on a set of 52 constrained NLS problems from the collections [62–64], and problems 2 and 3 referenced in [65]. For problems with variable dimensions, we set the number of variables to $n \in \{100, 500, 1000\}$. Thus, we have a total of 82 problem instances ranging from 2 to 1001 variables, with up to 2994 residuals and 999 constraints. Problems with inequality constraints were converted to

[1] Trust Region AUgmented nonLinear Least-squares Solver

the form (1) by adding slack variables. Only 5 of the 82 instances, HS14, HS22, HS23, HS32 and HS323, have a linear constraint kept explicit, a single equality in each case once slack variables are added. On all the others, the polyhedron (2) reduces to a box, on which the Cauchy path of Section 2.3 is the projected-gradient path of [40] and the ε-active bounds play no role. Section 4.3 therefore assesses the treatment of general polyhedra separately. For our tests, we set the relative criticality tolerance to $\omega_* = 10^{-5}$, the final test of Algorithm 1 being implemented as $\pi(x_K) \leq \omega_*(1+\pi(x_0))$ and each inner minimization stopping once its criticality measure falls below ω_K times one plus its value at the starting point of that minimization, the feasibility tolerance to $\eta_* = 10^{-6}$, the maximum number of outer iterations to 500 and the maximum number of inner iterations to 1000. The remaining parameters are set to $\mu_0 = 10$, $\tau = 10$, $\kappa_\omega = \beta_\omega = \eta = \omega = 1$, $\kappa_\eta = 0.1$, $\beta_\eta = 0.9$, $\kappa_{\text{hyb}} = 0.1$, $\delta_0 = 0.1$, $\varepsilon_{\max} = 10^{-3}$, and the trust-region constants $\alpha_1 = 0.25$, $\alpha_2 = 2.5$, $\eta_1 = 0.05$, $\eta_2 = 0.9$, $\gamma_1 = 0.0625$ recommended in [38, Section 17.1], in accordance with the conditions of Section 2.5.

To avoid any bias in computation time caused by Julia's just-in-time compilation, and for all solvers considered in this section, each instance was solved once to warm up, the result of that first solve being discarded, then solved three times with the minimum of the three timings reported. We present our results with Dolan–Moré performance profiles [66] produced with the `BenchmarkProfiles.jl` package [67]. We exclude from the profiles the problems on which the solvers considered for a given comparison do not all converge to the same solution, in the sense that

$$\frac{f_{\max} - f_{\min}}{1 + \max(|f_{\max}|, |f_{\min}|)} > 10^{-2}, \tag{55}$$

where $f_{\max}$ and $f_{\min}$ are respectively the maximum and minimum values of the objective obtained on a given instance. A similar discarding heuristic is also used in [29, 68].

4.1 Comparison of Hessian approximations

Our first set of experiments compares the behavior and performance of our algorithm depending on the Hessian approximation used during the inner minimization. We compare the GN approximation (27) against structured approximations of the form (28). Based on the structured secant equation (31), we implemented the SR1 update we discussed and the BFGS update that one can derive similarly. Both structured approximations were used in their plain and hybrid variants with the switching rule (35).

4.1.1 Profiles on the whole test set

We first compare these five strategies with respect to three metrics: computation time, number of outer iterations and number of inner iterations. The last metric coincides with the number of residual evaluations, since the trust-region mechanism evaluates the residuals exactly once per inner iteration. Results are reported in Figure 2.

One variant fails to reach a first-order critical point: the plain SR1 variant reaches the maximum number of outer iterations on problem HS373 and returns a point that

is only feasible. There, its iterates leave the basin of the solution at the second outer iteration and drift along a valley in which x_1 and x_2 grow without bound while the constraints remain satisfied, a situation excluded by Assumption 7. Problem HS23 and the instances of problems 5.13 and 5.14 from [64] with $n = 100$ were removed after evaluating the relative-gap criterion (55). The profiles are therefore built on the remaining 79 instances, for which all five variants agree on the solution.

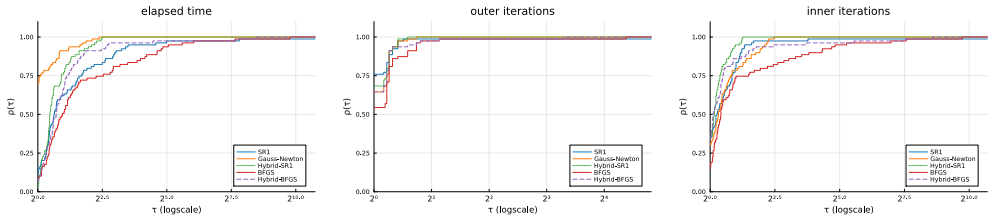


Fig. 2: Performance profiles comparing five different Hessian approximation strategies with respect to computation time (left), number of outer iterations (center), and number of inner iterations (right).

In terms of computation time, the GN approximation is the fastest and is also the cheapest per iteration because it requires no additional storage or update computation, and because our method mainly involves matrix-vector products. The GN approximation especially dominates on problems having sparse Jacobians and small residuals at the solution, since the second-order terms are still stored as a dense matrix. The Hybrid-SR1 variant is a close second.

Measured by outer iterations, the five strategies are almost indistinguishable, which is expected since the outer loop is governed by the augmented Lagrangian updates rather than by the Hessian model. The number of inner iterations, on the other hand, separates them clearly. There, the hybrid variants outperform their plain counterparts and the GN approximation alike. This is consistent with the discussion of the hybrid switching rule (35), which – as in the unconstrained case – correctly identifies situations where the Gauss–Newton approximation must be corrected and activates the structured quasi-Newton correction accordingly. Overall, Hybrid-SR1 appears to be the most robust variant, which also indicates that the SR1 approximation is well suited to handling negative curvature in the trust-region setting.

4.1.2 Profiles stratified by residual magnitude at the solution

To refine these observations and analyze separately the effectiveness of the approximate second-order correction and of the hybrid switching scheme, we stratify the previous profiles by the squared norm of the residuals at the solution, denoted $\|r_{\text{opt}}\|^2$. We distinguish four regimes: zero residuals ($\|r_{\text{opt}}\|^2 < 10^{-8}$, 18 instances), small residuals ($10^{-8} \leq \|r_{\text{opt}}\|^2 < 1$, 16 instances), medium residuals ($1 \leq \|r_{\text{opt}}\|^2 < 100$, 23 instances) and large residuals ($\|r_{\text{opt}}\|^2 \geq 100$, 22 instances). For this comparison, we retain the GN approximation and the SR1 update in its plain and hybrid variants. Figure 3 reports the profiles on the number of residual evaluations – equivalently, as

discussed above, on the number of inner iterations – and Figure 4 those on computation time.

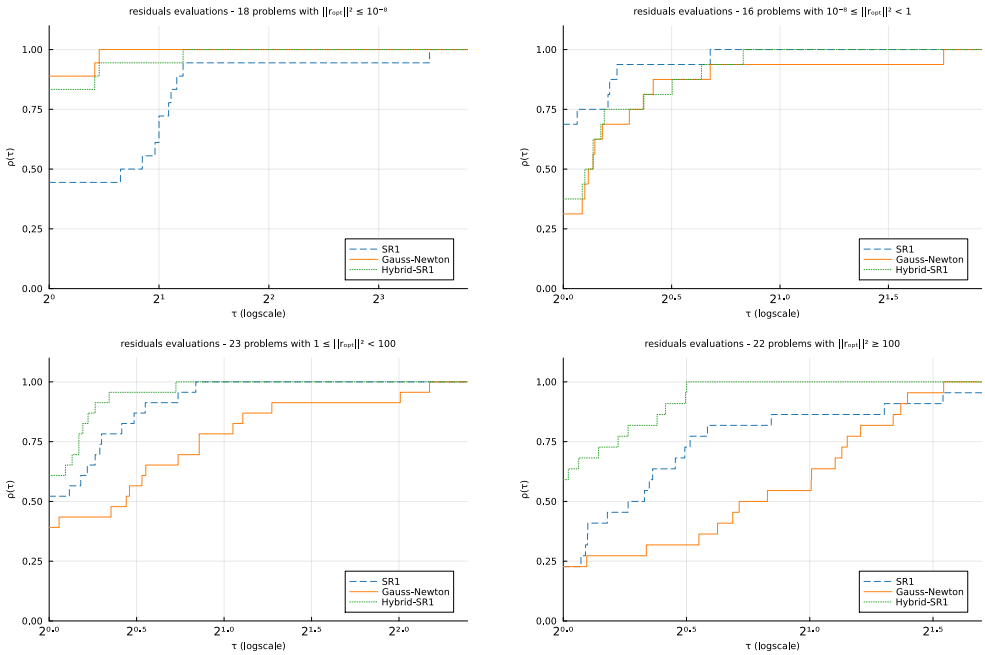


Fig. 3: Performance profiles on the number of residual evaluations, stratified by the squared norm of the residuals at the solution: zero residuals (top left), small residuals (top right), medium residuals (bottom left) and large residuals (bottom right).

The profiles on residual evaluations display a clear trend as the residual magnitude grows. The GN approximation performs better on the zero-residual instances but worse on the nonzero-residual ones, whereas Hybrid-SR1 is consistent across all residual magnitudes. The gap between the two even widens as the residuals get larger. The hybrid scheme therefore matches GN when the residuals vanish at the solution and takes a decisive lead as soon as the residuals are significant. This is precisely the behavior the switching rule is designed to produce.

The plain SR1 update stays close to its hybrid counterpart on the small- and medium-residual instances, and is even ahead on the former. The second-order correction is thus beneficial in itself, and the hybrid rule does not always exploit it fully in this intermediate regime. A plausible explanation is that the switching test (35) monitors the relative decrease of the augmented Lagrangian, a quantity influenced by both the residual size and the constraint violation, so that nonzero but still small residuals are not always identified as such. In the large-residual regime, by contrast, the hybrid rule is unambiguously the better strategy.

Computation time, however, favors the GN approximation in every stratum, even though the gap narrows as the residuals grow. The savings in residual evaluations

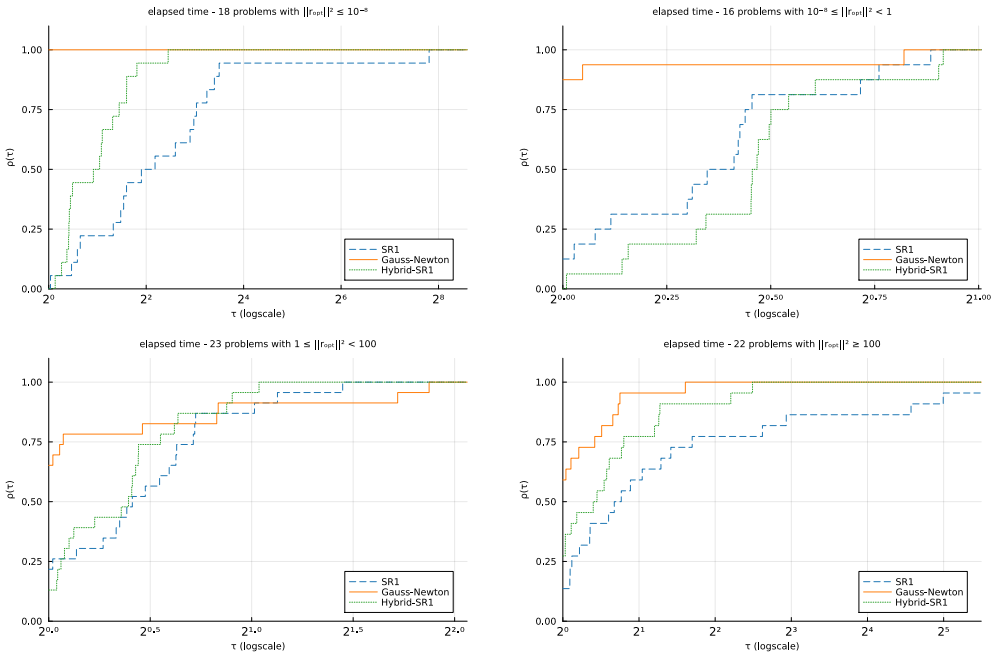


Fig. 4: Performance profiles on computation time, stratified by the squared norm of the residuals at the solution: zero residuals (top left), small residuals (top right), medium residuals (bottom left) and large residuals (bottom right).

achieved by the hybrid scheme therefore do not compensate for the additional cost of forming and using a dense second-order correction in the subsequent linear-algebra computations. This is largely due to how the test set is built. For all the problems considered, residuals are available by direct computation and their evaluation is consequently very cheap. In many applications, and in particular when the residuals come from a simulation code or from the solution of a system of ordinary differential equations, their evaluation is the dominant computational cost. In such settings, the profiles of Figure 3 are the relevant ones, and they show that the structured correction with hybrid switching does reduce the amount of work that actually matters.

To argue the latter observation, we ask at which evaluation cost the structured correction starts to pay off. Assume that one residual evaluation costs c more than it does here, every other cost being unchanged. The total time of a solver becomes

$$T(c) = T + c\, n_r^{\text{eval}}, \tag{56}$$

where T is the time we measured and n_r^{eval} the number of residual evaluations it performed. Comparing the GN approximation with Hybrid-SR1 on a given instance,

the two are equally fast at the break-even cost

$$c^\star = \frac{T_{\text{hyb}} - T_{\text{gn}}}{n_{\text{gn}}^{\text{eval}} - n_{\text{hyb}}^{\text{eval}}}, \tag{57}$$

which is meaningful when the hybrid variant performs fewer residual evaluations but takes more time, the only regime in which a crossover exists. Beyond $c^\star$, the hybrid variant is the faster of the two. This model keeps the iteration counts fixed and charges only the evaluations, so it describes a problem whose residuals are more expensive but whose solution path is unchanged.

Before applying it, we record in Table 1 how the evaluation counts of the two variants compare, stratum by stratum. The relative-gap criterion (55) is applied to the objective values reached by the GN approximation and Hybrid-SR1 alone, and it discards no instance. The pairwise counts are as informative as the geometric means. On the zero-residual instances the two variants perform exactly the same number of evaluations on 17 of the 18 instances, which confirms that the switching rule is inert when the correction is not needed and that it costs nothing there. The saving then grows with the residual magnitude. The Jacobian and gradient counts follow the same trend, which matters here, since an application whose residuals are expensive to evaluate usually has expensive derivatives as well.

		residual evaluations			geometric mean	
stratum	instances	fewer	equal	more	Jacobian	gradient
zero	18	0	17	1	1.04	1.04
small	16	4	6	6	0.95	0.96
medium	24	14	6	4	0.76	0.79
large	24	18	0	6	0.67	0.70

Table 1: Evaluation counts of Hybrid-SR1 relative to the GN approximation, by residual magnitude at the solution. The three middle columns count the instances on which Hybrid-SR1 performs fewer, exactly as many, or more residual evaluations than GN. The last two columns report the geometric mean of the ratio of the Jacobian and gradient evaluation counts, a value below one favoring Hybrid-SR1.

We now restrict the analysis to the 48 medium- and large-residual instances, since these are the ones on which the correction is meant to act. A crossover exists on 17 of them, those on which Hybrid-SR1 performs fewer residual evaluations but takes more time. On these instances, the break-even cost (57) ranges from 0.003 milliseconds at the first quartile to 1.18 milliseconds at the third quartile. We do not report the median, which is too sensitive to timing noise on so few instances: it changed by a factor of nearly three between two timed runs of the same code. In other words, an extra cost of about one millisecond per residual evaluation suffices for Hybrid-SR1 to become

the faster variant on three quarters of these instances. An evaluation that requires a simulation run or the integration of a system of ordinary differential equations is typically one to two orders of magnitude above that threshold.

Figure 5 shows the proportion of these instances on which Hybrid-SR1 is faster, as a function of the extra cost c. It starts at 31%, the proportion obtained with the times we measured, and saturates at two thirds as c increases, that is, on the 32 instances on which Hybrid-SR1 performs fewer residual evaluations. No evaluation cost can make it faster on the remaining ones. The conclusion we draw is therefore a conditional one, namely that the structured correction is the better choice on the majority of the problems with nonzero residuals as soon as the residuals are genuinely expensive, and that the threshold at which this happens is low enough to be reached by most applications in which the residuals come from a simulation.

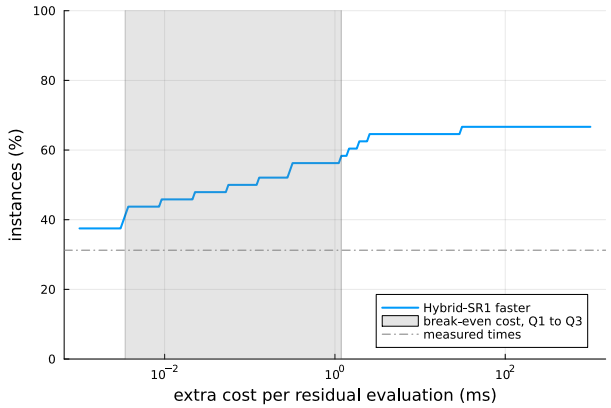


Fig. 5: Proportion of the 48 medium- and large-residual instances on which Hybrid-SR1 is faster than the GN approximation, as a function of the extra cost charged to every residual evaluation in model (56). The shaded band spans the break-even costs (57) between the first and third quartiles, and the dash-dotted line marks the proportion obtained with the measured times.

Based on these results, we retain the Hybrid-SR1 variant as the best overall strategy for TRAULLS, and use it in the comparisons that follow.

4.2 Comparison with IPOPT and Percival

We now compare TRAULLS (with the Hybrid-SR1 approximation) against the solvers IPOPT [68] and Percival [43]. The latter is also an augmented Lagrangian algorithm and has a native Julia implementation. Its inner minimization uses a Julia version of the TRON [46] algorithm – part of the `JSOSolvers.jl` package [69]. To benefit from the benchmarking facilities provided by the JuliaSmoothOptimizers (JSO)[2] ecosystem, we used the wrapper `NLPModelsIpopt.jl` [70]. This allows

[2] https://jso.dev

us to run both reference solvers on the same test set as TRAULLS – all models being encoded in `NLSProblems.jl` [71] – and to export the benchmark metrics using `SolverBenchmark.jl` [72]. All solvers shared the experimental setup and the tolerances described at the beginning of this section, and were additionally given a maximum CPU time of 10 minutes per instance, a limit consistent with the dimensions of the problems considered.

To make the comparison with IPOPT relevant, we set the parameters `nlp_scaling_method = none`, `dual_inf_tol = Inf`, `constr_viol_tol = Inf`, `compl_inf_tol = Inf`, `acceptable_iter = 0` and the maximum number of iterations to 1000. We also set `hessian_approximation = ''limited-memory''`, so that IPOPT builds its Hessian model with an L-BFGS update of memory 6 instead of exact second derivatives. This choice makes the comparison meaningful with respect to the derivative information used. For Percival, since its outer iteration is the same as that of Algorithm 1, we set its tolerances and parameters to the same values as ours. We note that Percival, through its TRON inner solver, uses exact Hessian-vector products, and is therefore the only solver of the three that accesses second derivatives.

For these tests, we compare the three solvers with respect to the computation time, the number of residual evaluations and the number of gradient evaluations. After evaluating criterion (55), we removed from the profiles the following nine instances: HS61, HS77, HS264, the instances with $n = 100$ of problems 5.1, 5.4, 5.13 and 5.14 from [64], and the instances with $n = 500$ and $n = 1000$ of problem 5.16 from the same collection. The profiles of Figure 6 are thus built on the remaining 73 instances. On these, TRAULLS and IPOPT converge on every instance, while Percival reaches the maximum CPU time on the two instances of problem 5.11 from [64] with $n = 500$ and $n = 1000$. As a side note, we also ran IPOPT with exact second derivatives, and this configuration failed on two instances, reaching the maximum number of iterations on HS27 and terminating with a restoration-phase failure on HS70.

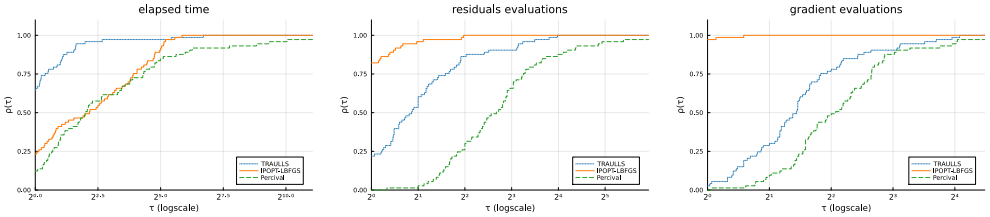


Fig. 6: Performance profiles comparing TRAULLS (Hybrid-SR1), IPOPT, and Percival on the test set, with respect to computation time (left), residual evaluations (center), and gradient evaluations (right).

In terms of computation time, TRAULLS is the fastest solver on more than 50% of the instances. This advantage is, however, strongly dependent on the problem size. For instance, restricting the comparison to IPOPT, TRAULLS is faster on all 41 instances with $n \leq 100$, but on only 10 of the 26 instances with $n \geq 500$, where our computation time is on average – in the geometric mean sense – 1.67 times that of

IPOPT. The margin observed on the smallest instances should be read with care, as both codes solve them in a few tens of milliseconds at most, with median times of 0.12 and 1.5 milliseconds respectively, so that the ratio there mostly reflects a lower fixed cost per solve rather than a difference in the optimization work performed. The large instances are where the gap is algorithmically meaningful, and there the cause is the dense storage and manipulation of the structured second-order correction, whose cost grows quadratically with the number of variables while the rest of the method only involves matrix-vector products. The example of problem 5.12 from [64] with $n = 1000$ illustrates that point: Hybrid-SR1 requires 142 inner iterations against 354 for the GN approximation, yet takes 1.93 seconds against 1.41. This is the limitation we discuss in the conclusion, and it is the reason a limited-memory form of the structured update is the natural next step for large-scale problems.

Regarding the evaluation metrics, IPOPT dominates both profiles – with a larger gap on gradients than on residuals – and this dominance decomposes into two distinct effects. TRAULLS performs on average 1.89 times as many residual evaluations as IPOPT, which reflects a genuine difference in the number of steps needed. On top of that, the two methods do not evaluate gradients at the same rate, since our trust-region scheme performs one gradient evaluation per inner iteration, whereas the line search in IPOPT reuses a single gradient across the trial points of a backtracking sequence. Part of the gap is thus structural, inherent to the globalization strategy rather than to the quality of our Hessian model.

These results should be read with the role of each solver in mind. IPOPT is a mature interior-point implementation, and it is included here as an external reference point rather than as the target of our contribution. The comparison that isolates what this work proposes is the one with Percival, which follows the same augmented Lagrangian framework, is run with the same tolerances and parameters, and differs from TRAULLS essentially in that it treats the problem as a generic nonlinear program. There, the advantage of TRAULLS is unambiguous and holds on all three metrics. On the 71 retained instances on which Percival converges, it takes in geometric mean 4.28 times as long as TRAULLS, and performs 2.94 times as many residual evaluations and 1.47 times as many gradient evaluations. On the two others, it reaches the time limit of 10 minutes, whereas TRAULLS converges in less than 0.1 second. Also, unlike the comparison with IPOPT, this advantage does not rest on the smallest instances; it is moreover obtained even though TRAULLS only approximates second-order information, whereas Percival evaluates exact Hessian-vector products. Within a common algorithmic framework, exploiting the least-squares structure through the Gauss–Newton term and its structured correction therefore leads to a clear improvement, which is the claim this paper makes. That TRAULLS is at the same time faster than IPOPT on most of the test set, while requiring more function evaluations, situates our implementation as a competitive solver whose remaining bottleneck – the dense second-order correction on large instances – is identified and addressable.

4.3 Linear constraints kept explicit

To assess the treatment of general polyhedra, we solved 15 problems with linear constraints in two ways: with the linear constraints kept explicit, as in (1), and with the

same constraints passed to the augmented Lagrangian as nonlinear constraints. Besides the five instances mentioned at the beginning of this section, the set contains HS42, HS49 and HS50, whose linear constraints were handled by the augmented Lagrangian in Sections 4.1 and 4.2, and problems HS21, HS48, HS51, HS52 and HS53 from [62] and HS224 and HS268 from [63], with the residuals of [71]. Once slack variables are added, the explicit formulations have up to 10 variables and 5 linear constraints. Table 2 reports the iteration counts of Hybrid-SR1 under both formulations and the violation of the linear constraints at the final point.

			outer iterations		inner iterations		linear violation	
problem	n	m	E	P	E	P	E	P
HS14	3	1	8	6	12	10	8×10^{-16}	1×10^{-8}
HS21	3	1	4	6	13	15	0	0
HS22	4	1	6	5	9	8	0	9×10^{-7}
HS23	7	1	6	7	23	69	0	0
HS32	4	1	5	7	7	9	1×10^{-15}	6×10^{-7}
HS42	4	1	6	6	15	16	0	6×10^{-7}
HS48	5	2	4	6	12	12	9×10^{-16}	1×10^{-7}
HS49	5	2	4	4	17	10	3×10^{-14}	1×10^{-11}
HS50	5	3	5	5	8	7	2×10^{-15}	1×10^{-7}
HS51	5	3	5	7	13	12	2×10^{-16}	9×10^{-7}
HS52	5	3	1	6	2	6	5×10^{-15}	1×10^{-7}
HS53	5	3	1	6	3	8	1×10^{-14}	2×10^{-7}
HS224	4	2	1	5	4	10	1×10^{-14}	1×10^{-7}
HS268	10	5	2	7	2	8	2×10^{-14}	2×10^{-11}
HS323	4	1	6	6	14	18	0	0

Table 2: Linear constraints kept explicit (E) or passed to the augmented Lagrangian (P), with Hybrid-SR1: number n of variables of the explicit formulation, slack variables included, number m of linear constraints, numbers of outer and inner iterations, and largest violation of the linear constraints at the final point.

Both formulations reach the same solution on 14 of the 15 problems. Kept explicit, the linear constraints hold to machine precision at every iterate, whereas the augmented Lagrangian only enforces them up to the feasibility tolerance. The explicit treatment also needs fewer iterations, about 40% fewer outer and 30% fewer inner iterations in geometric mean, the largest gains occurring when all the constraints are linear, as on HS52, HS53 and HS224, which are then solved within a single outer iteration. The exception is HS268, whose criticality measure at the starting point is large, about 7×10^3: the relative stopping test is then satisfied after two outer iterations of the explicit formulation, before the minimum is reached, whereas the penalized formulation gets closer to it during the additional outer iterations needed to reach feasibility.

4.4 Reproducibility

Every number reported in this section can be regenerated from the version of the repository tagged `coap-2026`. The `benchmark` folder contains a Julia environment which pins every package involved to the version used for the numerical experiments. The script `run_experiments.jl` runs the solvers under the protocol described at the beginning of this section and writes one result file per solver, recording for each instance the objective value reached, the elapsed time, the evaluation counters, the iteration counts and the termination status. Those files are the ones shipped in `results`, together with the commit of the solver code that produced them, so that the profiles and the tables can be rebuilt without running the solvers again. The script `make_figures.jl` produces the performance profiles, which can also be explored in the notebook `profiles.ipynb`, and `residual_cost_analysis.jl` the break-even analysis of Section 4.1. The script `linear_constraints/run_linear_constraints.jl` produces the results of Section 4.3, which only involve iteration counts. The input models for TRAULLS are given in the `traulls_models` folder.

5 Conclusion

In this paper, we have presented an algorithm able to solve a class of problems, namely NLS objectives under nonlinear equalities combined with bounds and linear equalities kept explicit, for which no dedicated method was available. Our method fully exploits the structure of the problem through the AL reformulation and a structured Hessian approximation. The resulting implementation is faster and more robust than a traditional AL solver, and remains competitive in computation time with a state-of-the-art solver for nonlinear programming. Despite this promising performance, the main limitation for large-scale applications lies in the dense storage of the structured second-order correction of the Hessian, in addition to the subsequent computations that become prohibitive as the number of variables grows. A limited-memory variant of the structured update, following [73], would be a natural feature to implement. To tackle problems with a large number of residuals, another avenue of research could be to combine the AL framework with an adaptive sampling strategy applied to the residuals, lowering the dimensions of the inner minimization subproblems. This would extend to the constrained case techniques currently investigated for unconstrained problems [74].

Acknowledgements. The first author is supported by Mitacs and Hydro-Québec through grants IT40867 and IT38322. The authors would also like to thank the Natural Sciences and Engineering Research Council of Canada (NSERC), which supports the second author through a Discovery Grant, and the Institute for Data Valorization (IVADO).

Appendix A Cauchy point computation

For the sake of clarity, we omit the iteration index during the rest of this appendix. We describe the procedure used to compute the first local minimizer of the quadratic

$q(s) = \frac{1}{2}\langle s, Hs\rangle + \langle g, s\rangle$ along a piecewise linear path $s(t)$, $t \geq 0$, which coincides with the projected gradient path $P_\Omega[x - tg] - x$ when Ω is a box. This is an adaptation of Algorithm 17.3.1 from [38] to the case where linear equalities are present. Over a general polyhedron, the active sets along the projected gradient path need not be nested, and the path below is instead defined by nested working sets, a bound being kept fixed once it is reached. For each $t \geq 0$, the projected step $s(t)$ satisfies the constraints

$$As(t) = 0, \qquad s^{(l)} \leq s(t) \leq s^{(u)},$$

where matrix A is $m \times n$ with $m < n$ and vectors $s^{(l)}$, $s^{(u)}$ are iteration-dependent bounds having finite components. The breakpoints

$$0 = t_0 < t_1 < \ldots < t_p,$$

correspond to the successive scalars at which at least one component of $s(t)$ satisfies a bound with equality. Assuming A has full row rank, there are $n - m$ degrees of freedom remaining and thus at most $n - m$ breakpoints before the projected gradient path is constant. We recall the recursive expression

$$s(t) = -(t - t_i)z_i + s(t_i),$$

for $t \in [t_i, t_{i+1})$ and with the reduced gradient z_i given by (42), the tangent space being now defined by the working set $\mathcal{A}(t_i)$.

The working set $\mathcal{A}(0)$ is selected among the ε-active bounds at x, $\varepsilon \geq 0$ being the threshold of Section 2.1 ($\varepsilon = 0$ corresponds to the bounds active at x). The corresponding variables are fixed at their current values, and the least-squares estimates θ of their multipliers are given by the decomposition (12) of g, which the projection provides at no extra cost. A bound is kept in $\mathcal{A}(0)$ if its multiplier has the sign required at a first-order critical point, and freed otherwise. The freed bounds along which the resulting direction $-z_0$ points outwards are fixed again, and this is repeated until every freed bound is left along $-z_0$ or has a zero component in z_0 (iteration (41)). When $\varepsilon = 0$, this amounts to breakpoints of length zero. When $\varepsilon > 0$, a freed bound may lie at a positive distance from x, and it is fixed again explicitly. When Ω is a box and $\varepsilon = 0$, $\theta_i = g_i$ and this rule reduces to fixing the variables at a bound along which $-g$ points outwards. With this choice, $-z_0$ is a feasible descent direction with $\langle g, z_0\rangle = \|z_0\|^2$ and $\|z_0\| \geq \kappa_0\, \pi_\varepsilon(x, g)$ (Proposition 2 and Lemma 3).

To find the first local minimum of the scalar function $\psi(t) = q(s(t))$, we successively study each interval $[t_i, t_{i+1})$ to determine whether or not it contains a local minimizer. Assume we have not found a local minimizer on $[t_0, t_i)$ and thus look at the interval $[t_i, t_{i+1})$. The model along this arc can be written

$$\psi(t) = \frac{\psi_i''}{2}(\Delta t)^2 + \psi_i' \Delta t \tag{A1}$$

with $\psi_i'' = \langle z_i, Hz_i \rangle$, $\psi_i' = -\langle s(t_i), Hz_i \rangle - \langle g, z_i \rangle$ and $\Delta t = t - t_i$. Different cases can occur depending on the values of the slope ψ_i' and the curvature ψ_i''. First, if

$$\psi_i' \geq 0,$$

we stop at t_i. This is the required minimizer, except in the degenerate case where $\psi_i' = 0$ and $\psi_i'' < 0$. Stopping there does not affect the decrease guarantee, which only involves the first segment of the path, on which ψ' is negative unless $z_0 = 0$. In floating-point arithmetic, we also stop when $\|z_i\|_\infty \leq 10\,\epsilon_{\mathrm{M}} \|g\|_\infty$, where ϵ_{M} is the relative machine precision. The direction z_i is then made of rounding errors, and the ratio $-\psi_i'/\psi_i''$ formed from it is meaningless and may even be negative. Next, if

$$\psi_i' < 0 \text{ and } \psi_i'' > 0,$$

the quadratic (A1) has a strict minimizer at

$$t_i - \frac{\psi_i'}{\psi_i''}.$$

If the latter belongs to the interval of interest, i.e.,

$$t_i - \psi_i'/\psi_i'' < t_{i+1},$$

then it is the required minimizer. In other cases, the minimizer is at or beyond t_{i+1}. To prepare for the study of the next interval, we first need to find the next breakpoint, given by the smallest scalar $t_{i+1} > t_i$ such that, at $s(t_{i+1})$, one of the free components hits one of its bounds. By introducing

$$\delta_i^- = \min_{(z_i)_j > 0} \left\{ \frac{(s(t_i))_j - s_j^{(l)}}{(z_i)_j} \right\}, \qquad \delta_i^+ = \min_{(z_i)_j < 0} \left\{ \frac{(s(t_i))_j - s_j^{(u)}}{(z_i)_j} \right\},$$

the next breakpoint is given by

$$t_{i+1} = t_i + \delta_i, \tag{A2}$$

with $\delta_i = \min\left(\delta_i^-, \delta_i^+\right)$. We then add the corresponding variable index to the list of fixed components, compute the next reduced gradient z_{i+1} and finally update the slope and curvature of the model along the next interval. A last termination case can occur. Since A is of rank m, at most $n - m$ bounds can become active, so if we never find a local minimum, the procedure still ends whenever it reaches the last breakpoint t_p. Indeed, past this breakpoint, the model along the projected gradient path is constant so we can return the last accumulated step $s(t_p)$ as the Cauchy step. Note that, in this case, it is also the total step, because no more variables can be modified. The finiteness of the box $\left[s^{(l)}, s^{(u)}\right]$ guarantees that all components of the projected step $s(t)$ are fixed at one of their bounds whenever t passes the last breakpoint. The procedure for the Cauchy step computation is outlined in Algorithm 4.

Algorithm 4 Cauchy step computation

Require: Hessian H, gradient g, bounds on the direction $s^{(l)}$, $s^{(u)}$

Fix the variables of the ε-active bounds at x, compute their multiplier estimates θ, free those of the wrong sign, and compute the direction z_0, fixing again the freed bounds along which $-z_0$ points outwards
Set **found** $\leftarrow$ **false**
Set $t_0 \leftarrow 0$, $s^{(0)} \leftarrow 0$ and initialize counter $i \leftarrow 0$
repeat
    $\psi_i' \leftarrow -\langle s^{(i)}, Hz_i \rangle - \langle g, z_i \rangle$, $\psi_i'' \leftarrow \langle z_i, Hz_i \rangle$
    Find the next breakpoint t_{i+1} by (A2)
    if $\psi_i' \geq 0$ or $\|z_i\|_\infty \leq 10\,\epsilon_{\mathrm{M}} \|g\|_\infty$ **then**
        Set $s^C \leftarrow s^{(i)}$
        found $\leftarrow$ **true**
    else if $\psi_i' < 0$ and $\psi_i'' > 0$ and $t_i - \psi_i'/\psi_i'' < t_{i+1}$ **then**
        Set $\Delta t \leftarrow -\psi_i'/\psi_i''$
        Set $s^C \leftarrow s^{(i)} - \Delta t z_i$, **found** $\leftarrow$ **true**
    else
        Set $\Delta t \leftarrow t_{i+1} - t_i$
        Compute the next direction z_{i+1}
        Set $s^{(i+1)} \leftarrow s^{(i)} - \Delta t z_i$
    end if
    Increment $i \leftarrow i + 1$
    if $|\mathcal{A}(t_i)| = n - m$ **then**
        Set $s^C \leftarrow s^{(i)}$, **found** $\leftarrow$ **true**
    end if
until found
return s^C

References

[1] Dennis Jr, J.E., Schnabel, R.B.: Numerical Methods for Unconstrained Optimization and Nonlinear Equations. Classics in Applied Mathematics. SIAM, Philadelphia, PA, USA (1996). https://doi.org/10.1137/1.9781611971200

[2] Nocedal, J., Wright, S.J.: Numerical Optimization, 2nd edn. Springer Series in Operations Research and Financial Engineering. Springer, New York, NY, USA (2006). https://doi.org/10.1007/978-0-387-40065-5

[3] Björck, Å.: Numerical Methods for Least Squares Problems, 2nd edn. SIAM, Philadelphia, PA, USA (2024). https://doi.org/10.1137/1.9781611977950

[4] Levenberg, K.: A method for the solution of certain non-linear problems in least squares. Quarterly of Applied Mathematics **2**, 164–168 (1944) https://doi.org/10.1090/qam/10666

[5] Marquardt, D.W.: An algorithm for least squares estimation of non-linear parameters. Journal of the Society for Industrial and Applied Mathematics **11**, 431–441 (1963) https://doi.org/10.1137/0111030

[6] Bellavia, S., Gratton, S., Riccietti, E.: A Levenberg-Marquardt method for large nonlinear least-squares problems with dynamic accuracy in functions and gradients. Numerische Mathematik **140**, 791–825 (2018) https://doi.org/10.1007/s00211-018-0977-z

[7] Bergou, E.H., Diouane, Y., Kungurtsev, V., Royer, C.W.: A nonmonotone matrix-free algorithm for nonlinear equality-constrained least-squares problems. SIAM Journal on Scientific Computing **43**(5), 743–766 (2021) https://doi.org/10.1137/20M1349138

[8] Dennis Jr, J.E., Gay, D.M., Welsch, R.E.: An adaptive nonlinear least-squares algorithm. ACM Transactions on Mathematical Software **7**(3), 348–368 (1981) https://doi.org/10.1145/355958.355965

[9] Dennis Jr, J.E., Martinez, H.J., Tapia, R.A.: Convergence theory for the structured BFGS secant method with an application to nonlinear least squares. Journal of Optimization Theory and Applications **61**(2), 161–178 (1989) https://doi.org/10.1007/BF00962795

[10] Al-Baali, M., Fletcher, R.: Variational methods for non-linear least-squares. The Journal of the Operational Research Society **36**(5), 405–421 (1985) https://doi.org/10.1057/jors.1985.68

[11] Fletcher, R., Xu, C.: Hybrid methods for nonlinear least squares. IMA Journal of Numerical Analysis **7**(3), 371–389 (1987) https://doi.org/10.1093/imanum/7.3.371

[12] Zhou, W., Chen, X.: Global convergence of a new hybrid Gauss-Newton structured BFGS method for nonlinear least squares problems. SIAM Journal on Optimization **20**(5), 2422–2441 (2010) https://doi.org/10.1137/090748470

[13] Bartholomew-Biggs, M.C.: The estimation of the hessian matrix in nonlinear least-squares problems with non-zero residuals. Mathematical Programming **12**, 67–80 (1977) https://doi.org/10.1007/BF01593770

[14] Betts, J.T.: Solving the nonlinear least square problem: Application of a general method. Journal of Optimization Theory and Applications **18**, 469–483 (1976) https://doi.org/10.1007/BF00932656

[15] Huschens, J.: On the use of product structure in secant methods for nonlinear least squares problems. SIAM Journal on Optimization **4**(1), 108–129 (1994) https://doi.org/10.1137/0804005

[16] Yabe, H., Takahashi, T.: Factorized quasi-Newton methods for nonlinear least squares problems. Mathematical Programming **51**, 75–100 (1991) https://doi.org/10.1007/BF01586927

[17] Zhang, J.Z., Chen, L.H., Deng, N.Y.: A family of scaled factorized Broyden-like methods for nonlinear least squares problems. SIAM Journal on Optimization **10**(4), 1163–1179 (2000) https://doi.org/10.1137/S1052623498345300

[18] Lukšan, L., Matonoha, C., Vlček, J.: Hybrid methods for nonlinear least squares problems. Technical Report 1246, Institute of Computer Science, Academy of Sciences of the Czech Republic (2019)

[19] Grenier, M., Gagnon, J., Mercier, C., Richard, J.: Short-term load forecasting at Hydro-Québec TransÉnergie. In: 2006 IEEE Power Engineering Society General Meeting, Montreal, QC, Canada (2006). https://doi.org/10.1109/PES.2006.1709029

[20] Delbos, F., Gilbert, J.C., Glowinski, R., Sinoquet, D.: Constrained optimization in seismic reflection tomography: a Gauss-Newton augmented Lagrangian approach. Geophysical Journal International **164**, 670–684 (2006) https://doi.org/10.1111/j.1365-246X.2005.02729.x

[21] Pu, H., Chen, Z., Wang, B., Xia, W.: Constrained least squares algorithms for nonlinear unmixing of hyperspectral imagery. IEEE Transactions on Geoscience and Remote Sensing **53**(3), 1287–1303 (2015) https://doi.org/10.1109/TGRS.2014.2336858

[22] Bellavia, S., Macconi, M., Morini, B.: STRSCNE: A scaled trust-region solver for constrained nonlinear equations. Computational Optimization and Applications **28**, 31–50 (2004)

[23] Bellavia, S., Gondzio, J., Morini, B.: Computational experience with numerical methods for nonnegative least-squares problems. Numerical Linear Algebra with Applications **18**(3), 363–385 (2011)

[24] Porcelli, M.: On the convergence of an inexact Gauss-Newton trust-region method for nonlinear least-squares problems with simple bounds. Optimization Letters **7**, 447–465 (2013) https://doi.org/10.1007/s11590-011-0430-z

[25] Gonçalves, M.L.N., Menezes, T.C.: Gauss–Newton methods with approximate projections for solving constrained nonlinear least squares problems. Journal of Complexity **58**, 101459 (2020) https://doi.org/10.1016/j.jco.2020.101459

[26] Li, Z., Osborne, M., Prvan, T.: Adaptive algorithm for constrained least-squares problems. Journal of Optimization Theory and Applications **114**, 423–441 (2002) https://doi.org/10.1023/A:1016043919978

[27] Tjoa, I.B., Biegler, L.T.: Simultaneous solution and optimization strategies for parameter estimation of differential-algebraic equation systems. Industrial & Engineering Chemistry Research **30**(2), 376–385 (1991) https://doi.org/10.1021/ie00050a015

[28] Mahdavi-Amiri, N., Bartels, R.H.: Constrained nonlinear least squares: An exact penalty approach with projected structured quasi-Newton updates. ACM Transactions on Mathematical Software **15**(3), 220–242 (1989) https://doi.org/10.1145/66888.66891

[29] Orban, D., Siqueira, A.S.: A regularization method for constrained nonlinear least squares. Computational Optimization and Applications **76**(3), 961–989 (2020) https://doi.org/10.1007/s10589-020-00201-2

[30] Tapia, R.A.: On secant updates for use in general constrained optimization. Mathematics of Computation **51**(183), 181–202 (1988) https://doi.org/10.2307/2008585

[31] Hestenes, M.R.: Multiplier and gradient methods. Journal of Optimization Theory and Applications **4**, 303–320 (1969) https://doi.org/10.1007/BF00927673

[32] Powell, M.J.D.: A method for nonlinear constraints in minimization problems. In: Fletcher, R. (ed.) Optimization, pp. 283–298. Academic Press, London (1969)

[33] Rockafellar, R.T.: The multiplier method of Hestenes and Powell applied to convex programming. Journal of Optimization Theory and Applications **12**(6), 555–562 (1973) https://doi.org/10.1007/BF00934777

[34] Conn, A.R., Gould, N.I.M., Toint, P.L.: Global convergence of a class of trust region algorithms for optimization with simple bounds. SIAM Journal on Numerical Analysis **25**(2), 433–460 (1988) https://doi.org/10.1137/0725029

[35] Conn, A.R., Gould, N.I.M., Toint, P.L.: A globally convergent augmented Lagrangian algorithm for optimization with general constraints and simple bounds. SIAM Journal on Numerical Analysis **28**(2), 545–572 (1991) https://doi.org/10.1137/0728030

[36] Conn, A.R., Gould, N.I.M., Sartenaer, A., Toint, P.L.: Global convergence of a class of trust region algorithms for optimization using inexact projections on convex constraints. SIAM Journal on Optimization **3**(1), 164–221 (1993) https://doi.org/10.1137/0803009

[37] Conn, A.R., Gould, N.I.M., Sartenaer, A., Toint, P.L.: Convergence properties of an augmented Lagrangian algorithm for optimization with a combination of general equality and linear constraints. SIAM Journal on Optimization **6**(3), 674–703 (1996) https://doi.org/10.1137/S1052623493251463

[38] Conn, A.R., Gould, N.I.M., Toint, P.L.: Trust Region Methods. SIAM, Philadelphia, PA, USA (2000). https://doi.org/10.1137/1.9780898719857

[39] Conn, A.R., Gould, N.I.M., Toint, P.L.: LANCELOT: A Fortran Package for Large-Scale Nonlinear Optimization (Release A). Springer Series in Computational Mathematics. Springer, Heidelberg (1992). https://doi.org/10.1007/978-3-662-12211-2

[40] Conn, A.R., Gould, N.I.M., Toint, P.L.: Testing a class of methods for solving minimization problems with simple bounds on the variables. Mathematics of Computation **50**(182), 399–430 (1988) https://doi.org/10.1090/S0025-5718-1988-0929544-3

[41] Andreani, R., Birgin, E.G., Martinez, J.M., Schuverdt, M.L.: On augmented Lagrangian methods with general lower-level constraints. SIAM Journal on Optimization **18**(4), 1286–1309 (2008) https://doi.org/10.1137/060654797

[42] Gill, P.E., Robinson, D.P.: A primal-dual augmented Lagrangian. Computational Optimization and Applications **51**, 1–25 (2012) https://doi.org/10.1007/s10589-010-9339-1

[43] Arreckx, S., Lambe, A., Martins, J.R.R.A., Orban, D.: A matrix-free augmented Lagrangian algorithm with application to large-scale structural design optimization. Optimization and Engineering **17**, 359–384 (2016) https://doi.org/10.1007/s11081-015-9287-9

[44] Curtis, F.E., Jiang, H., Robinson, D.P.: An adaptive augmented Lagrangian method for large-scale constrained optimization. Mathematical Programming, Series A **152**, 201–245 (2015) https://doi.org/10.1007/s10107-014-0784-y

[45] Moré, J.J., Toraldo, G.: On the solution of large quadratic programming problems with bound constraints. SIAM Journal on Optimization **1**(1), 93–113 (1991) https://doi.org/10.1137/0801008

[46] Lin, C.-J., Moré, J.J.: Newton's method for large bound-constrained optimization problems. SIAM Journal on Optimization **9**(4), 1100–1127 (1999) https://doi.org/10.1137/S1052623498345075

[47] Gould, N.I.M., Hribar, M.E., Nocedal, J.: On the solution of equality constrained quadratic programming problems arising in optimization. SIAM Journal on Scientific Computing **23**(4), 1376–1395 (2001) https://doi.org/10.1137/S1064827598345667

[48] Murtagh, B.A., Saunders, M.A.: Large-scale linearly constrained optimization. Mathematical Programming **14**, 41–72 (1978) https://doi.org/10.1007/BF01588950

[49] Toint, P.L., Tuyttens, D.: LSNNO, a FORTRAN subroutine for solving large-scale nonlinear network optimization problems. ACM Transactions on Mathematical Software **18**(3), 308–328 (1992) https://doi.org/10.1145/131766.131771

[50] Calamai, P.H., Moré, J.J.: Projected gradient methods for linearly constrained problems. Mathematical Programming **39**(1), 93–116 (1987)

[51] Zoutendijk, G.: Methods of Feasible Directions: A Study in Linear and Non-Linear Programming. Elsevier, Amsterdam (1960)

[52] Wolfe, P.: On the convergence of gradient methods under constraint. IBM Journal of Research and Development **16**(4), 407–411 (1972)

[53] Rosen, J.B.: The gradient projection method for nonlinear programming. Part I. Linear constraints. Journal of the Society for Industrial and Applied Mathematics **8**(1), 181–217 (1960)

[54] Topkis, D.M., Veinott, A.F. Jr.: On the convergence of some feasible direction algorithms for nonlinear programming. SIAM Journal on Control **5**(2), 268–279 (1967)

[55] Bertsekas, D.P.: Projected Newton methods for optimization problems with simple constraints. SIAM Journal on Control and Optimization **20**(2), 221–246 (1982)

[56] Bezanson, J., Edelman, A., Karpinski, S., Shah, V.B.: Julia: A fresh approach to numerical computing. SIAM Review **59**(1), 65–98 (2017) https://doi.org/10.1137/141000671

[57] Dennis Jr, J.E., Walker, H.F.: Convergence theorems for least-change secant update methods. SIAM Journal on Numerical Analysis **18**(6), 949–987 (1981) https://doi.org/10.1137/0718067

[58] Conn, A.R., Gould, N.I.M., Toint, P.L.: Convergence of quasi-Newton matrices generated by the symmetric rank one update. Mathematical Programming **50**, 177–195 (1991) https://doi.org/10.1007/BF01594934

[59] Golub, G.H., Van Loan, C.F.: Matrix Computations, 4th edn. JHU Press, Baltimore, MD, USA (2013). https://doi.org/10.56021/9781421407944

[60] McCormick, G.P.: Anti-zig-zagging by bending. Management Science **15**(5), 315–320 (1969)

[61] Hoffman, A.J.: On approximate solutions of systems of linear inequalities. Journal of Research of the National Bureau of Standards **49**(4), 263–265 (1952)

[62] Hock, W., Schittkowski, K.: Test Examples for Nonlinear Programming Codes. Lecture Notes in Economics and Mathematical Systems, vol. 187. Springer,

Heidelberg (1981). https://doi.org/10.1007/978-3-642-48320-2

[63] Schittkowski, K.: More Test Examples for Nonlinear Programming Codes. Lecture Notes in Economics and Mathematical Systems. Springer, Heidelberg (1987). https://doi.org/10.1007/978-3-642-61582-5

[64] Lukšan, L., Vlček, J.: Sparse and partially separable test problems for unconstrained and equality constrained optimization. Technical Report 767, Institute of Computer Science, Academy of Sciences of the Czech Republic (1999)

[65] Biegler, L.T., Nocedal, J., Schmid, C., Ternet, D.: Numerical experience with a reduced hessian method for large scale constrained optimization. Computational Optimization and Applications **15**, 45–67 (2000) https://doi.org/10.1023/A:1008723031056

[66] Dolan, E.D., Moré, J.J.: Benchmarking optimization software with performance profiles. Mathematical Programming **91**(2), 201–213 (2002) https://doi.org/10.1007/s101070100263

[67] Orban, D.: BenchmarkProfiles.jl. Zenodo (2019). https://doi.org/10.5281/zenodo.4630955

[68] Wächter, A., Biegler, L.T.: On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. Mathematical Programming, Series A **106**, 25–57 (2006) https://doi.org/10.1007/s10107-004-0559-y

[69] Migot, T., Monnet, D., Orban, D., Soares Siqueira, A.: JSOSolvers.jl: Unconstrained and bound-constrained optimization solvers. Journal of Open Source Software **11**(117), 9467 (2026) https://doi.org/10.21105/joss.09467

[70] Soares Siqueira, A., Orban, D.: NLPModelsIpopt.jl. Zenodo (2026). https://doi.org/10.5281/zenodo.19420624

[71] Soares Siqueira, A., Orban, D.: NLSProblems.jl. Zenodo (2019). https://doi.org/10.5281/zenodo.4605405

[72] Soares Siqueira, A., Orban, D.: SolverBenchmark.jl. Zenodo (2020). https://doi.org/10.5281/zenodo.3948381

[73] Byrd, R.H., Nocedal, J., Schnabel, R.B.: Representations of quasi-Newton matrices and their use in limited memory methods. Mathematical Programming **63**, 129–156 (1994) https://doi.org/10.1007/BF01582063

[74] Bellavia, S., Malaspina, G., Morini, B.: A variable dimension sketching strategy for nonlinear least-squares. SIAM Journal on Scientific Computing **48**(3), 1206–1234 (2026) https://doi.org/10.1137/25M175980X